

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SalesMates CRM</title>

  <style>
    :root {
      --bg: #090d16;
      --surface: #111827;
      --surface-card: #1f2937;
      --sidebar: #070a13;
      --text: #f3f4f6;
      --muted: #9ca3af;
      --border: #374151;
      --primary: #3b82f6;
      --primary-hover: #2563eb;
      --success: #10b981;
      --warning: #f59e0b;
      --danger: #ef4444;
      --info: #60a5fa;
      --radius: 12px;
      --shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.3), 0 2px 4px -1px rgba(0, 0, 0, 0.2);
    }

    * { box-sizing: border-box; }

    body {
      margin: 0;
      min-width: 320px;
      background: var(--bg);
      color: var(--text);
      font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe
UI", sans-serif;
    }

    /* Scrollbar Estilizada Premium */
    ::-webkit-scrollbar {
      width: 8px;
      height: 8px;
    }
    ::-webkit-scrollbar-track {
      background: var(--bg);
    }
    ::-webkit-scrollbar-thumb {
      background: #374151;
      border-radius: 99px;
```

```

}
::-webkit-scrollbar-thumb:hover {
  background: #4b5563;
}

button, input, select, textarea { font: inherit; }
button { border: 0; cursor: pointer; }
input, select, textarea {
  width: 100%;
  outline: none;
  border: 1px solid var(--border);
  border-radius: 8px;
  background: var(--surface-card);
  color: var(--text);
  transition: border .2s, box-shadow .2s;
}

input:focus, select:focus, textarea:focus {
  border-color: var(--primary);
  box-shadow: 0 0 3px rgba(59, 130, 246, 0.25);
}

textarea { resize: vertical; }

.app { min-height: 100vh; }

/* SIDEBAR */
.sidebar {
  position: fixed;
  z-index: 10;
  top: 0;
  bottom: 0;
  left: 0;
  width: 240px;
  display: flex;
  flex-direction: column;
  padding: 24px 16px;
  background: var(--sidebar);
  border-right: 1px solid var(--border);
  color: white;
}

.brand {
  display: flex;
  align-items: center;
  margin-bottom: 24px;
  padding: 0 8px;
}

```

```
.brand-logo {  
  color: white;  
  letter-spacing: -.05em;  
  font-size: 22px;  
  line-height: 1;  
}
```

```
.brand-logo strong { font-weight: 800; color: var(--primary); }  
.brand-logo span { color: #f3f4f6; font-weight: 300; }
```

```
.menu-label {  
  padding: 12px 8px 6px;  
  color: #4b5563;  
  font-size: 11px;  
  font-weight: 700;  
  letter-spacing: .05em;  
  text-transform: uppercase;  
}
```

```
.nav-button {  
  width: 100%;  
  display: flex;  
  align-items: center;  
  gap: 12px;  
  padding: 10px 12px;  
  margin: 2px 0;  
  border-radius: 8px;  
  color: #9ca3af;  
  background: transparent;  
  text-align: left;  
  font-size: 13px;  
  font-weight: 600;  
  transition: .15s ease;  
}
```

```
.nav-button:hover {  
  color: white;  
  background: rgba(255,255,255,.05);  
}
```

```
.nav-button.active {  
  color: white;  
  background: var(--primary);  
}
```

```
.nav-icon {  
  width: 16px;
```

```

    text-align: center;
    font-size: 15px;
    font-style: normal;
}

.sidebar-footer {
    margin-top: auto;
    padding-top: 16px;
    border-top: 1px solid var(--border);
}

.user-box {
    display: flex;
    align-items: center;
    gap: 10px;
    padding: 8px;
    color: #d1d5db;
    font-size: 12px;
}

.avatar, .small-avatar {
    display: grid;
    place-items: center;
    border-radius: 50%;
    background: var(--primary);
    color: white;
    font-weight: 700;
}

.avatar {
    width: 32px;
    height: 32px;
    flex: 0 0 auto;
}

.user-box strong { color: white; font-size: 13px; }
.user-box small { color: #6b7280; }

/* MAIN */
.main {
    min-height: 100vh;
    margin-left: 240px;
    padding: 24px 32px;
}

.view { display: none; }
.view.active { display: block; }

```

```

.topbar {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 24px;
}

.page-title h1 {
  margin: 0;
  font-size: 24px;
  letter-spacing: -.03em;
}

.page-title p {
  margin: 6px 0 0;
  color: var(--muted);
  font-size: 14px;
}

.actions, .filters, .inline-actions {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 8px;
}

.button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  padding: 8px 14px;
  border-radius: 8px;
  font-size: 13px;
  font-weight: 600;
  transition: .15s ease;
}

.button-primary { color: white; background: var(--primary); }
.button-primary:hover { background: var(--primary-hover); }
.button-secondary { border: 1px solid var(--border); background: var(--surface-card); color:
var(--text); }
.button-secondary:hover { background: #374151; }
.button-danger { color: white; background: var(--danger); }
.button-success { color: white; background: var(--success); }
.button-small { padding: 6px 10px; font-size: 12px; }

```

```

.icon-button {
  width: 34px;
  height: 34px;
  display: inline-grid;
  place-items: center;
  border: 1px solid var(--border);
  border-radius: 8px;
  background: var(--surface-card);
  color: var(--text);
  font-size: 14px;
}

.icon-button:hover { background: #374151; }
.icon-button.danger { color: var(--danger); }
.icon-button.danger:hover { background: rgba(239, 68, 68, 0.2); }

.panel, .metric-card {
  border: 1px solid var(--border);
  border-radius: var(--radius);
  background: var(--surface);
  box-shadow: var(--shadow);
}

.panel { padding: 20px; }

.panel-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 16px;
}

.panel-header h2 { margin: 0; font-size: 15px; letter-spacing: -.02em; color: white; }
.panel-subtitle { margin: 2px 0 0; color: var(--muted); font-size: 12px; }

.dashboard-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 16px;
  margin-bottom: 24px;
}

.metric-card { padding: 16px; background: var(--surface); }
.metric-label { color: var(--muted); font-size: 12px; font-weight: 600; }
.metric-value {
  margin: 8px 0 4px;
  font-size: 22px;
}

```

```
font-weight: 700;
letter-spacing: -.03em;
color: white;
}

.metric-note { color: var(--muted); font-size: 11px; }
.metric-note.danger { color: var(--danger); }
.metric-note.success { color: var(--success); }

.content-grid {
  display: grid;
  grid-template-columns: 1.5fr 1fr;
  gap: 20px;
}

.progress-row { margin: 16px 0; }
.progress-top {
  display: flex;
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 6px;
  font-size: 12px;
}

.progress-track {
  height: 6px;
  overflow: hidden;
  border-radius: 99px;
  background: #1f2937;
}

.progress-bar {
  height: 100%;
  border-radius: inherit;
  background: var(--primary);
}

.badge {
  display: inline-flex;
  align-items: center;
  width: fit-content;
  padding: 3px 8px;
  border-radius: 999px;
  font-size: 11px;
  font-weight: 600;
  white-space: nowrap;
}
```

```
.badge-green { color: #34d399; background: rgba(16, 185, 129, 0.15); border: 1px solid
rgba(16, 185, 129, 0.3); }
.badge-red { color: #f87171; background: rgba(239, 68, 68, 0.15); border: 1px solid
rgba(239, 68, 68, 0.3); }
.badge-blue { color: #60a5fa; background: rgba(59, 130, 246, 0.15); border: 1px solid
rgba(59, 130, 246, 0.3); }
.badge-yellow { color: #fbbf24; background: rgba(245, 158, 11, 0.15); border: 1px solid
rgba(245, 158, 11, 0.3); }
.badge-gray { color: #d1d5db; background: rgba(75, 85, 99, 0.15); border: 1px solid
rgba(75, 85, 99, 0.3); }
```

```
.activity-item {
  display: flex;
  align-items: flex-start;
  gap: 10px;
  padding: 12px 0;
  border-bottom: 1px solid var(--border);
}
```

```
.activity-item:last-child { border-bottom: 0; }
```

```
.activity-icon {
  width: 28px;
  height: 28px;
  display: grid;
  flex: 0 0 auto;
  place-items: center;
  border-radius: 6px;
  background: var(--surface-card);
  font-size: 13px;
}
```

```
.activity-info { min-width: 0; flex: 1; }
.activity-info strong, .activity-info small { display: block; }
.activity-info strong {
  overflow: hidden;
  font-size: 12px;
  text-overflow: ellipsis;
  white-space: nowrap;
  color: white;
}
```

```
.activity-info small {
  margin-top: 2px;
  color: var(--muted);
  font-size: 11px;
}
```



```
/* FILTERS */
.toolbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 20px;
}

.search-input {
  min-width: 240px;
  padding: 8px 12px;
  font-size: 13px;
}

.filter, .view-toggle {
  width: auto;
  padding: 8px 12px;
  border: 1px solid var(--border);
  border-radius: 8px;
  color: var(--text);
  background: var(--surface-card);
  font-size: 12px;
  font-weight: 500;
}

.filter.active, .view-toggle.active {
  border-color: var(--primary);
  color: white;
  background: var(--primary);
}

/* SALES RHYTHM */
.conversion-list { display: grid; gap: 10px; }

.conversion-row {
  padding: 12px;
  border: 1px solid var(--border);
  border-radius: 8px;
  background: var(--surface-card);
}

.conversion-top {
  display: flex;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 6px;
  font-size: 12px;
}
```

```

}

.conversion-top strong { font-size: 12px; color: white; }
.conversion-meta {
  margin-top: 6px;
  color: var(--muted);
  font-size: 10px;
}

/* PIPELINE MANAGER */
.pipeline-manager {
  display: none;
  margin-bottom: 16px;
  padding: 16px;
  border: 1px dashed var(--border);
  border-radius: var(--radius);
  background: var(--surface);
}

.pipeline-manager.open { display: block; }

.stage-editor {
  display: flex;
  align-items: stretch;
  gap: 8px;
  overflow-x: auto;
  padding: 6px 0;
}

.stage-editor-card {
  min-width: 200px;
  padding: 10px;
  border: 1px solid var(--border);
  border-radius: 8px;
  background: var(--surface-card);
}

.stage-editor-card strong {
  display: block;
  margin-bottom: 6px;
  overflow: hidden;
  font-size: 12px;
  text-overflow: ellipsis;
  white-space: nowrap;
  color: white;
}

.stage-editor-actions {

```

```

display: flex;
align-items: center;
gap: 4px;
}

.stage-add-between {
width: 32px;
min-width: 32px;
height: 32px;
align-self: center;
display: grid;
place-items: center;
border: 1px dashed var(--muted);
border-radius: 50%;
background: var(--surface-card);
color: var(--text);
font-size: 16px;
}

.stage-add-between:hover {
color: white;
background: var(--primary);
border-color: var(--primary);
}

/* KANBAN HORIZONTAL COMPACTO */
.kanban-board {
display: flex;
gap: 12px;
overflow-x: auto;
padding-bottom: 16px;
align-items: flex-start;
flex-wrap: nowrap;
}

.kanban-column {
width: 230px;
min-width: 230px;
flex-shrink: 0;
min-height: calc(100vh - 240px);
padding: 10px;
border: 1px solid var(--border);
border-radius: var(--radius);
background: var(--surface);
display: flex;
flex-direction: column;
gap: 8px;
}

```

```
.column-header {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  padding-bottom: 6px;  
  border-bottom: 2px solid var(--border);  
  font-size: 11px;  
  font-weight: 700;  
}
```

```
.column-title {  
  max-width: 150px;  
  overflow: hidden;  
  text-overflow: ellipsis;  
  white-space: nowrap;  
  text-transform: uppercase;  
  font-size: 11px;  
  color: white;  
}
```

```
.column-total {  
  display: block;  
  margin-top: 2px;  
  color: var(--muted);  
  font-size: 10px;  
  font-weight: 500;  
}
```

```
.column-count {  
  padding: 1px 5px;  
  border-radius: 4px;  
  color: var(--text);  
  background: var(--surface-card);  
  font-size: 10px;  
}
```

/* PIPEDRIVE DARK DEAL CARD */

```
.deal-card {  
  padding: 10px;  
  border: 1px solid var(--border);  
  border-radius: 8px;  
  background: var(--surface-card);  
  box-shadow: var(--shadow);  
  cursor: grab;  
  transition: all 0.15s ease;  
  display: flex;  
  flex-direction: column;
```

```
    gap: 4px;
}

.deal-card:hover {
  border-color: var(--primary);
  transform: translateY(-1px);
  box-shadow: 0 4px 12px rgba(59, 130, 246, 0.15);
}

.deal-card h3 {
  margin: 0;
  font-size: 12px;
  font-weight: 700;
  color: white;
  line-height: 1.3;
}

.deal-card p {
  margin: 0;
  color: var(--muted);
  font-size: 10px;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.deal-value {
  font-size: 11px;
  font-weight: 700;
  color: white;
  margin-top: 2px;
}

.deal-card-tags {
  display: flex;
  flex-wrap: wrap;
  gap: 3px;
  margin-top: 2px;
}

.deal-card-tag {
  font-size: 8px;
  padding: 1px 4px;
  background: rgba(255,255,255,0.06);
  border-radius: 4px;
  color: #d1d5db;
  font-weight: 600;
}
```

```
.deal-card-footer {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  margin-top: 6px;  
  padding-top: 6px;  
  border-top: 1px solid var(--border);  
}
```

```
.deal-card-owner {  
  font-size: 9px;  
  color: var(--muted);  
  display: flex;  
  align-items: center;  
  gap: 4px;  
}
```

```
.activity-dot {  
  width: 8px;  
  height: 8px;  
  border-radius: 50%;  
  display: inline-block;  
}
```

```
.activity-dot.green { background-color: var(--success); }  
.activity-dot.red { background-color: var(--danger); }  
.activity-dot.yellow { background-color: var(--warning); }  
.activity-dot.gray { background-color: #4b5563; }
```

```
.pipeline-list {  
  overflow: hidden;  
  border: 1px solid var(--border);  
  border-radius: var(--radius);  
  background: var(--surface);  
}
```

/* BARRA DE CHEVRONS INTERATIVA */

```
.chevron-bar {  
  display: flex;  
  overflow-x: auto;  
  gap: 2px;  
  margin: 16px 0;  
  padding: 4px 0;  
}
```

```
.chevron-step {  
  position: relative;  
  flex: 1;
```

```

min-width: 110px;
text-align: center;
background: #1f2937;
color: #9ca3af;
padding: 8px 12px 8px 20px;
font-size: 10px;
font-weight: 600;
clip-path: polygon(0% 0%, calc(100% - 10px) 0%, 100% 50%, calc(100% - 10px) 100%,
0% 100%, 10px 50%);
cursor: pointer;
transition: all 0.2s;
white-space: nowrap;
text-overflow: ellipsis;
overflow: hidden;
}

.chevron-step:first-child {
clip-path: polygon(0% 0%, calc(100% - 10px) 0%, 100% 50%, calc(100% - 10px) 100%,
0% 100%);
padding-left: 12px;
border-radius: 4px 0 0 4px;
}

.chevron-step:last-child {
clip-path: polygon(0% 0%, 100% 0%, 100% 100%, 0% 100%, 10px 50%);
border-radius: 0 4px 4px 0;
}

.chevron-step:hover {
background: #374151;
color: white;
}

.chevron-step.completed {
background: #065f46;
color: #34d399;
}

.chevron-step.active {
background: var(--primary);
color: white;
}

/* TABLES */
.table-wrap {
overflow: auto;
border: 1px solid var(--border);
border-radius: var(--radius);

```

```

    background: var(--surface);
}

table { width: 100%; border-collapse: collapse; }

th, td {
    padding: 12px 16px;
    border-bottom: 1px solid var(--border);
    text-align: left;
    font-size: 12px;
}

th {
    color: var(--muted);
    background: var(--surface-card);
    font-size: 11px;
    letter-spacing: .05em;
    text-transform: uppercase;
    font-weight: 600;
}

tr:last-child td { border-bottom: 0; }
tbody tr:hover { background: var(--surface-card); }

.link-button {
    padding: 0;
    color: var(--primary);
    background: transparent;
    font-weight: 600;
    text-align: left;
}

.empty {
    padding: 32px;
    color: var(--muted);
    text-align: center;
    font-size: 13px;
}

/* CALENDAR */
.agenda-layout {
    display: grid;
    grid-template-columns: 280px 1fr;
    gap: 20px;
}

.calendar-mini { padding: 16px; }

```



```
.calendar-navigation {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 12px;
}

.calendar-title {
  font-size: 13px;
  font-weight: 700;
  text-transform: capitalize;
}

.weekdays, .days-grid {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  gap: 2px;
}

.weekdays span {
  padding-bottom: 4px;
  color: var(--muted);
  text-align: center;
  font-size: 10px;
  font-weight: 700;
}

.calendar-day {
  aspect-ratio: 1;
  position: relative;
  display: grid;
  place-items: center;
  border-radius: 6px;
  background: transparent;
  color: var(--text);
  font-size: 11px;
}

.calendar-day:hover { background: var(--surface-card); }
.calendar-day.empty-day { pointer-events: none; }
.calendar-day.today { border: 1px solid white; font-weight: 700; }
.calendar-day.selected { color: white; background: var(--primary); }

.calendar-day.has-activity::after {
  content: "";
  position: absolute;
  bottom: 3px;
  width: 4px;
}
```

```
height: 4px;  
border-radius: 50%;  
background: var(--info);  
}
```

```
.calendar-day.selected.has-activity::after { background: white; }
```

```
.agenda-summary {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 6px;  
  margin-top: 16px;  
}
```

```
.agenda-summary div {  
  padding: 8px;  
  border-radius: 8px;  
  background: var(--surface-card);  
  font-size: 10px;  
}
```

```
.agenda-summary strong {  
  display: block;  
  margin-top: 2px;  
  font-size: 14px;  
  color: white;  
}
```

```
.agenda-main { min-width: 0; }
```

```
.agenda-heading {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  gap: 12px;  
  margin-bottom: 12px;  
}
```

```
.agenda-date-label {  
  font-size: 14px;  
  font-weight: 700;  
  text-transform: capitalize;  
}
```

```
.activity-chart {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 10px;
```

```
    margin-bottom: 16px;
}
```

```
.chart-card {
  padding: 12px;
  border: 1px solid var(--border);
  border-radius: 8px;
  background: var(--surface);
}
```

```
.chart-card span {
  color: var(--muted);
  font-size: 10px;
  font-weight: 600;
}
```

```
.chart-card strong {
  display: block;
  margin: 4px 0 6px;
  font-size: 18px;
  color: white;
}
```

```
/* TEAM */
```

```
.team-grid {
  display: grid;
  grid-template-columns: 1.5fr 1fr;
  gap: 20px;
}
```

```
.team-row {
  display: grid;
  grid-template-columns: 1.5fr repeat(4, 1fr);
  gap: 8px;
  align-items: center;
  padding: 10px 0;
  border-bottom: 1px solid var(--border);
  font-size: 12px;
}
```

```
.team-row:last-child { border-bottom: 0; }
```

```
.team-person {
  display: flex;
  align-items: center;
  gap: 8px;
  font-weight: 600;
}
```

```

.small-avatar {
  width: 24px;
  height: 24px;
  background: var(--surface-card);
  font-size: 9px;
}

/* MODALS */
.modal-backdrop {
  display: none;
  position: fixed;
  z-index: 100;
  inset: 0;
  padding: 24px;
  overflow: auto;
  background: rgba(0,0,0,.75);
}

.modal-backdrop.open { display: block; }
#activityModal { z-index: 130; }

.modal {
  width: min(950px, 100%);
  margin: 20px auto;
  overflow: hidden;
  border-radius: 12px;
  border: 1px solid var(--border);
  background: var(--surface);
  box-shadow: var(--shadow);
}

.modal.small-modal { width: min(500px, 100%); }

.modal-header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 16px;
  padding: 16px 24px;
  border-bottom: 1px solid var(--border);
}

.modal-header h2 {
  margin: 0;
  font-size: 18px;
  letter-spacing: -.02em;
  color: white;
}

```

```

}

.modal-header p {
  margin: 4px 0 0;
  color: var(--muted);
  font-size: 12px;
}

.modal-body { padding: 20px 24px; }

.close-button {
  width: 32px;
  height: 32px;
  display: grid;
  flex: 0 0 auto;
  place-items: center;
  border-radius: 6px;
  background: var(--surface-card);
  color: var(--text);
  font-size: 18px;
}

.close-button:hover { background: #374151; }

.form-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 12px;
}

.field {
  display: flex;
  flex-direction: column;
  gap: 4px;
}

.field.full { grid-column: 1 / -1; }
.field label { font-size: 11px; font-weight: 700; color: #9ca3af; }

.field input, .field select, .field textarea {
  padding: 8px 10px;
  font-size: 13px;
}

.field textarea { min-height: 80px; }

.modal-actions {
  display: flex;

```

```
flex-wrap: wrap;
justify-content: flex-end;
gap: 8px;
margin-top: 18px;
}

.modal-content {
display: grid;
grid-template-columns: 300px 1fr;
min-height: 480px;
}

.deal-details {
padding: 16px 20px;
border-right: 1px solid var(--border);
background: var(--sidebar);
}

.detail-title {
margin: 16px 0 6px;
color: var(--muted);
font-size: 10px;
font-weight: 700;
letter-spacing: .05em;
text-transform: uppercase;
}

.detail-value {
margin-bottom: 10px;
font-size: 12px;
line-height: 1.4;
}

.detail-value strong { font-size: 11px; color: white; }
.detail-actions { display: grid; gap: 6px; }

.timeline { padding: 20px; background: var(--surface); }

.timeline-event {
position: relative;
padding: 0 0 16px 20px;
border-left: 1px solid var(--border);
}

.timeline-event::before {
content: "";
position: absolute;
top: 1px;
```

```
left: -4px;
width: 7px;
height: 7px;
border: 1px solid white;
border-radius: 50%;
background: var(--primary);
}
```

```
.timeline-event strong, .timeline-event small { display: block; }
.timeline-event strong { font-size: 12px; color: white; }
```

```
.timeline-event small {
margin-top: 2px;
color: var(--muted);
font-size: 11px;
}
```

```
.whatsapp-box {
padding: 10px;
margin-top: 12px;
border: 1px solid rgba(16, 185, 129, 0.3);
border-radius: 8px;
background: rgba(16, 185, 129, 0.08);
}
```

```
.whatsapp-box strong {
display: block;
margin-bottom: 4px;
color: #34d399;
font-size: 11px;
}
```

```
.whatsapp-box p {
margin: 0 0 8px;
color: #a7f3d0;
font-size: 10px;
line-height: 1.4;
}
```

```
@media (max-width: 1000px) {
.sidebar { width: 64px; padding: 16px 8px; }
.brand { justify-content: center; padding: 0; }
.brand-logo { display: none; }
.menu-label, .nav-button span:not(.nav-icon), .user-box > div:last-child { display: none; }
.nav-button { justify-content: center; padding: 10px; }
.sidebar-footer { display: flex; justify-content: center; }
.user-box { padding: 4px; }
.main { margin-left: 64px; }
```

```

.dashboard-grid { grid-template-columns: repeat(2, 1fr); }
.content-grid, .team-grid { grid-template-columns: 1fr; }
}

@media (max-width: 768px) {
  .main { margin-left: 0; padding: 16px; }

  .sidebar {
    position: static;
    width: 100%;
    min-height: auto;
    flex-direction: row;
    align-items: center;
    padding: 8px;
    overflow-x: auto;
  }

  .brand { margin: 0 12px 0 0; }
  .menu-label, .sidebar-footer { display: none; }

  .nav-button {
    width: auto;
    flex: 0 0 auto;
    margin: 0;
    padding: 8px;
  }

  .nav-button span:not(.nav-icon) { display: none; }

  .topbar, .toolbar, .agenda-heading {
    align-items: flex-start;
    flex-direction: column;
  }

  .topbar .actions { width: 100%; }
  .topbar .actions .button { flex: 1; }
  .dashboard-grid, .activity-chart { grid-template-columns: 1fr; }
  .agenda-layout, .modal-content { grid-template-columns: 1fr; }
  .deal-details { border-right: 0; border-bottom: 1px solid var(--border); }
  .form-grid { grid-template-columns: 1fr; }
  .field.full { grid-column: auto; }
  .modal-backdrop { padding: 8px; }
  .modal { margin: 4px auto; }
  .search-input { min-width: 100%; }
}
</style>
</head>

```



```

<body>
<div class="app">
  <aside class="sidebar">
    <div class="brand" title="SalesMates">
      <div class="brand-logo">
        <strong>Sales</strong><span>Mates</span>
      </div>
    </div>

    <div class="menu-label">Operação</div>

    <button class="nav-button active" data-view="dashboard">
      <i class="nav-icon">△</i><span>Meu painel</span>
    </button>

    <button class="nav-button" data-view="pipeline">
      <i class="nav-icon">▮</i><span>Funil comercial</span>
    </button>

    <button class="nav-button" data-view="activities">
      <i class="nav-icon">⌚</i><span>Agenda</span>
    </button>

    <button class="nav-button" data-view="forecast">
      <i class="nav-icon">↗</i><span>Forecast</span>
    </button>

    <div class="menu-label">Gestão</div>

    <button class="nav-button" data-view="team">
      <i class="nav-icon">👤</i><span>Visão de equipe</span>
    </button>

    <button class="nav-button" data-view="reports">
      <i class="nav-icon">📊</i><span>Relatórios</span>
    </button>

    <button class="nav-button" data-view="settings">
      <i class="nav-icon">⚙️</i><span>Configurações</span>
    </button>

    <div class="sidebar-footer">
      <div class="user-box">
        <div class="avatar">V</div>
        <div>
          <strong>Victor</strong><br />
          <small>Administrador</small>
        </div>
      </div>
    </div>
  </aside>
</div>

```

```
</div>
</div>
</aside>
```

```
<main class="main">
  <!-- DASHBOARD -->
  <section class="view active" id="dashboard">
    <div class="topbar">
      <div class="page-title">
        <h1>Olá, Victor.</h1>
        <p>Uma visão rápida da sua operação comercial.</p>
      </div>

      <div class="actions">
        <button class="button button-secondary"
onclick="showView('activities')">Agenda</button>
        <button class="button button-primary" onclick="openActivityModal()">+ Nova
atividade</button>
      </div>
    </div>

    <div class="dashboard-grid">
      <article class="metric-card">
        <div class="metric-label">Concluídas hoje</div>
        <div class="metric-value" id="todayCompleted">0</div>
        <div class="metric-note">Atividades finalizadas</div>
      </article>

      <article class="metric-card">
        <div class="metric-label">Pendentes</div>
        <div class="metric-value" id="pendingActivities">0</div>
        <div class="metric-note">Próximas ações agendadas</div>
      </article>

      <article class="metric-card">
        <div class="metric-label">Precisam de atenção</div>
        <div class="metric-value" id="overdueActivities">0</div>
        <div class="metric-note danger">Atividades vencidas</div>
      </article>

      <article class="metric-card">
        <div class="metric-label">Forecast ponderado</div>
        <div class="metric-value" id="weeklyForecast">R$ 0</div>
        <div class="metric-note success">Oportunidades em aberto</div>
      </article>
    </div>

    <div class="content-grid">
```

```

<section class="panel">
  <div class="panel-header">
    <div>
      <h2>Ritmo comercial</h2>
      <p class="panel-subtitle">Conversão atual entre as etapas do funil comercial.</p>
    </div>
    <button class="button button-secondary button-small"
onclick="showView('pipeline')">Ver funil</button>
  </div>

  <div id="conversionDashboard" class="conversion-list"></div>
</section>

<section class="panel">
  <div class="panel-header">
    <div>
      <h2>Próximas atividades</h2>
      <p class="panel-subtitle">Priorize o que move o funil.</p>
    </div>
    <button class="button button-secondary button-small"
onclick="showView('activities')">Ver agenda</button>
  </div>

  <div id="dashboardActivities"></div>
</section>
</div>
</section>

<!-- PIPELINE -->
<section class="view" id="pipeline">
  <div class="topbar">
    <div class="page-title">
      <h1>Funil comercial</h1>
      <p>Gerencie oportunidades, responsáveis, etapas e evolução da negociação.</p>
    </div>

    <div class="actions">
      <button class="button button-secondary" onclick="togglePipelineManager()">⚙️
Editar funil</button>
      <button class="button button-primary" onclick="openDealModal()">+ Novo
negócio</button>
    </div>
  </div>

  <section class="pipeline-manager" id="pipelineManager">
    <div class="panel-header">
      <div>
        <h2>Editar etapas do funil</h2>

```

<p class="panel-subtitle">Você pode criar, editar, excluir e alterar a posição de cada etapa.</p>

</div>

<button class="close-button" onclick="togglePipelineManager()">x</button>

</div>

<div class="stage-editor" id="stageEditor"></div>

<div style="margin-top:14px;color:var(--muted);font-size:11px;line-height:1.5">

Ao excluir uma etapa, os negócios dela serão movidos para a etapa anterior. É necessário manter ao menos duas etapas no funil.

</div>

</section>

<div class="toolbar">

<div class="filters">

<input

class="search-input"

id="pipelineSearch"

placeholder="Empresa, contato, telefone ou e-mail..."

oninput="renderKanban()"

/>

<select class="filter" id="pipelineOwner" onchange="renderKanban()">

<option value="">Todos os responsáveis</option>

</select>

<select class="filter" id="pipelineActivity" onchange="renderKanban()">

<option value="">Todas as atividades</option>

<option value="overdue">Com atividade vencida</option>

<option value="none">Sem próxima atividade</option>

</select>

<select class="filter" id="pipelineStatus" onchange="renderKanban()">

<option value="open">Ativos</option>

<option value="lost">Perdidos</option>

<option value="churn">Churn</option>

<option value="won">Ganhos</option>

<option value="all">Todos os status</option>

</select>

<select class="filter" id="pipelineLossReason" onchange="renderKanban()">

<option value="">Todas as perdas</option>

<option>Preço</option>

<option>Sem retorno do cliente</option>

<option>Concorrência</option>

<option>Sem fit</option>

<option>Timing inadequado</option>

```

        <option>Outro</option>
    </select>
</div>

<div class="filters">
    <button class="view-toggle active" id="kanbanToggle"
onclick="setPipelineView('kanban')">≡ Kanban</button>
    <button class="view-toggle" id="listToggle" onclick="setPipelineView('list')">≡≡≡
Lista</button>
</div>
</div>

<div id="kanbanBoard" class="kanban-board"></div>
<div id="pipelineList" class="pipeline-list" style="display:none"></div>
</section>

<!-- ACTIVITIES -->
<section class="view" id="activities">
    <div class="topbar">
        <div class="page-title">
            <h1>Agenda comercial</h1>
            <p>Organize atividades por data, período, responsável e prioridade.</p>
        </div>

        <div class="actions">
            <button class="button button-secondary" onclick="goToday()">Hoje</button>
            <button class="button button-primary" onclick="openActivityModal()">+ Nova
atividade</button>
        </div>
    </div>

    <div class="agenda-layout">
        <aside class="panel calendar-mini">
            <div class="calendar-navigation">
                <button class="icon-button" onclick="changeCalendarMonth(-1)"></button>
                <div class="calendar-title" id="calendarTitle"></div>
                <button class="icon-button" onclick="changeCalendarMonth(1)"></button>
            </div>

            <div class="weekdays">

<span>D</span><span>S</span><span>T</span><span>Q</span><span>Q</span><span>S</span><span>S</span>
n>S</span><span>S</span>
            </div>

            <div class="days-grid" id="calendarDays"></div>

            <div class="agenda-summary">

```

```

    <div>Agendadas<strong id="agendaScheduled">0</strong></div>
    <div>Vencidas<strong id="agendaOverdue">0</strong></div>
    <div>Concluídas<strong id="agendaCompleted">0</strong></div>
  </div>
</aside>

<section class="agenda-main">
  <div class="activity-chart">
    <div class="chart-card">
      <span>Pendentes no período</span>
      <strong id="chartPending">0</strong>
      <div class="progress-track"><div id="chartPendingBar" class="progress-bar"
style="width:0%;background:#3b82f6"></div></div>
    </div>

    <div class="chart-card">
      <span>Vencidas no período</span>
      <strong id="chartOverdue">0</strong>
      <div class="progress-track"><div id="chartOverdueBar" class="progress-bar"
style="width:0%;background:#ef4444"></div></div>
    </div>

    <div class="chart-card">
      <span>Concluídas no período</span>
      <strong id="chartCompleted">0</strong>
      <div class="progress-track"><div id="chartCompletedBar" class="progress-bar"
style="width:0%;background:#10b981"></div></div>
    </div>
  </div>

  <div class="toolbar">
    <div class="filters">
      <select class="filter" id="activityOwnerFilter" onchange="renderActivities()">
        <option value="">Todos responsáveis</option>
      </select>

      <select class="filter" id="activityTypeFilter" onchange="renderActivities()">
        <option value="">Todos os tipos</option>
        <option>Ligação</option>
        <option>WhatsApp</option>
        <option>Reunião</option>
        <option>Visita</option>
        <option>Proposta</option>
        <option>Follow-up</option>
        <option>Tarefa</option>
      </select>

      <select class="filter" id="activityStatusFilter" onchange="renderActivities()">

```

```

        <option value="">Todos os status</option>
        <option value="Agendada">Agendadas</option>
        <option value="Atrasada">Vencidas</option>
        <option value="Concluída">Concluídas</option>
    </select>
</div>

    <input class="search-input" id="activitySearch" placeholder="Buscar atividade ou
negócio..." oninput="renderActivities()" />
</div>

<div class="agenda-heading">
    <strong class="agenda-date-label" id="agendaDateLabel"></strong>

    <div class="filters">
        <button class="view-toggle active" id="agendaDayToggle"
onclick="setAgendaView('day')">Dia</button>
        <button class="view-toggle" id="agendaWeekToggle"
onclick="setAgendaView('week')">Semana</button>
        <button class="view-toggle" id="agendaMonthToggle"
onclick="setAgendaView('month')">Mês</button>
        <button class="view-toggle" id="agendaYearToggle"
onclick="setAgendaView('year')">Ano</button>
        <button class="view-toggle" id="agendaListToggle"
onclick="setAgendaView('list')">Lista</button>
    </div>
</div>

<div class="table-wrap">
    <table>
        <thead>
            <tr>
                <th>Atividade</th>
                <th>Tipo</th>
                <th>Negócio</th>
                <th>Responsável</th>
                <th>Data</th>
                <th>Status</th>
                <th></th>
            </tr>
        </thead>
        <tbody id="activitiesTable"></tbody>
    </table>
</div>
</section>
</div>
</section>

```

```

<!-- FORECAST -->
<section class="view" id="forecast">
  <div class="topbar">
    <div class="page-title">
      <h1>Forecast comercial</h1>
      <p>Atualizado automaticamente conforme valor, etapa e probabilidade dos
negócios.</p>
    </div>

    <div class="actions">
      <select class="filter" id="forecastOwnerFilter" onchange="renderForecast()">
        <option value="">Toda a equipe</option>
      </select>
    </div>
  </div>

  <div class="dashboard-grid">
    <article class="metric-card">
      <div class="metric-label">Meta do período</div>
      <div class="metric-value" id="forecastGoal">R$ 80.000</div>
      <div class="metric-note">Meta mensal simulada</div>
    </article>

    <article class="metric-card">
      <div class="metric-label">Pipeline total</div>
      <div class="metric-value" id="grossForecast">R$ 0</div>
      <div class="metric-note">Negócios abertos com valor</div>
    </article>

    <article class="metric-card">
      <div class="metric-label">Forecast ponderado</div>
      <div class="metric-value" id="weightedForecast">R$ 0</div>
      <div class="metric-note">Valor ajustado pela probabilidade</div>
    </article>

    <article class="metric-card">
      <div class="metric-label">Gap para a meta</div>
      <div class="metric-value" id="goalGap">R$ 0</div>
      <div class="metric-note" id="goalGapNote">Receita a conquistar</div>
    </article>
  </div>

  <div class="content-grid">
    <section class="panel">
      <div class="panel-header">
        <div>
          <h2>Impacto por etapa</h2>

```


<p class="panel-subtitle">Conversão e previsão ponderada dentro do pipeline.</p>

</div>

</div>

<div id="forecastStages" class="conversion-list"></div>

</section>

<section class="panel">

<div class="panel-header">

<div>

<h2>Resumo do período</h2>

<p class="panel-subtitle">Indicadores gerados a partir das oportunidades ativas.</p>

</div>

</div>

<div id="forecastSummary"></div>

</section>

</div>

<section class="panel" style="margin-top:18px">

<div class="panel-header">

<div>

<h2>Oportunidades no forecast</h2>

<p class="panel-subtitle">Negócios sem valor não distorcem a projeção.</p>

</div>

0 sem valor

</div>

<div class="table-wrap">

<table>

<thead>

<tr>

<th>Negócio</th>

<th>Responsável</th>

<th>Etapa</th>

<th>Valor</th>

<th>Probabilidade</th>

<th>Forecast</th>

</tr>

</thead>

<tbody id="forecastTable"></tbody>

</table>

</div>

</section>

</section>

```

<!-- TEAM -->
<section class="view" id="team">
  <div class="topbar">
    <div class="page-title">
      <h1>Visão de equipe</h1>
      <p>Pipeline, produtividade, metas e atividades por responsável.</p>
    </div>

    <div class="actions">
      <select class="filter" id="teamPeriod" onchange="renderTeam()">
        <option value="month">Mês atual</option>
        <option value="week">Semana atual</option>
        <option value="all">Histórico completo</option>
      </select>
    </div>
  </div>

  <div class="dashboard-grid">
    <article class="metric-card">
      <div class="metric-label">Pipeline da equipe</div>
      <div class="metric-value" id="teamPipeline">R$ 0</div>
      <div class="metric-note">Oportunidades em aberto</div>
    </article>

    <article class="metric-card">
      <div class="metric-label">Atividades pendentes</div>
      <div class="metric-value" id="teamPending">0</div>
      <div class="metric-note">Ações planejadas</div>
    </article>

    <article class="metric-card">
      <div class="metric-label">Atividades vencidas</div>
      <div class="metric-value" id="teamOverdue">0</div>
      <div class="metric-note danger">Acompanhar com prioridade</div>
    </article>

    <article class="metric-card">
      <div class="metric-label">Receita ganha</div>
      <div class="metric-value" id="teamWon">R$ 0</div>
      <div class="metric-note success">Negócios fechados como ganho</div>
    </article>
  </div>

  <div class="team-grid">
    <section class="panel">
      <div class="panel-header">
        <div>
          <h2>Ranking comercial</h2>

```

```

        <p class="panel-subtitle">Pipeline, atividades, atrasos e forecast individual.</p>
    </div>
</div>

<div style="overflow:auto">
    <div id="teamRanking"></div>
</div>
</section>

<section class="panel">
    <div class="panel-header">
        <div>
            <h2>Alertas do time</h2>
            <p class="panel-subtitle">Atividades que precisam de acompanhamento.</p>
        </div>
    </div>

    <div id="teamAlerts"></div>
</section>
</div>
</section>

<!-- REPORTS -->
<section class="view" id="reports">
    <div class="topbar">
        <div class="page-title">
            <h1>Relatórios</h1>
            <p>Indicadores de produtividade, conversão, funil e receita.</p>
        </div>
    </div>
</div>

<div class="content-grid">
    <section class="panel">
        <div class="panel-header">
            <div>
                <h2>Conversão por etapa</h2>
                <p class="panel-subtitle">Dados calculados com base nas oportunidades
cadastradas.</p>
            </div>
        </div>
    </div>

    <div id="reportsConversion" class="conversion-list"></div>
</section>

<section class="panel">
    <div class="panel-header">
        <div>
            <h2>Motivos de perda</h2>

```

```

        <p class="panel-subtitle">Visibilidade sobre os motivos mais recorrentes.</p>
    </div>
</div>

<div id="lossReports"></div>
</section>
</div>
</section>

<!-- SETTINGS -->
<section class="view" id="settings">
    <div class="topbar">
        <div class="page-title">
            <h1>Configurações</h1>
            <p>Estrutura comercial, backup e controle da equipe.</p>
        </div>
    </div>

    <section class="panel">
        <div class="panel-header">
            <div>
                <h2>Metas da operação</h2>
                <p class="panel-subtitle">Defina o objetivo mensal comercial de vendas.</p>
            </div>
        </div>

        <div class="form-grid">
            <div class="field">
                <label>Meta mensal da operação (R$)</label>
                <input id="goalInput" type="number" min="0" value="80000" />
            </div>
        </div>

        <div class="modal-actions">
            <button class="button button-primary" onclick="saveGoal()">Salvar meta</button>
        </div>
    </section>

    <!-- BACKUP IMPORT / EXPORT -->
    <section class="panel" style="margin-top: 24px;">
        <div class="panel-header">
            <div>
                <h2>Backup dos Dados (Importar/Exportar)</h2>
                <p class="panel-subtitle">Exporte todos os seus dados do CRM para salvaguarda
ou importe um arquivo de backup previamente gerado.</p>
            </div>
        </div>
    </section>

```

```

<div class="modal-actions" style="justify-content: flex-start;">
  <button class="button button-secondary" onclick="exportBackup()"> ↓ Exportar
  Dados (JSON)</button>
  <button class="button button-secondary" onclick="triggerImportBackup()"> ↑
  Importar Dados (JSON)</button>
  <input type="file" id="backupFileInput" style="display: none;" accept=".json"
  onchange="importBackup(event)" />
</div>
</section>

```

```

<!-- GERENCIAR VENDEDORES -->
<section class="panel" style="margin-top: 24px;">
  <div class="panel-header">
    <div>
      <h2>Gerenciar Vendedores (Equipe)</h2>
      <p class="panel-subtitle">Adicione ou remova membros da equipe de vendas da
      sua conta.</p>
    </div>
  </div>

```

```

<div style="max-width: 500px; margin-bottom: 20px;">
  <div class="field">
    <label>Nome do Novo Vendedor</label>
    <div style="display: flex; gap: 8px;">
      <input id="newOwnerName" placeholder="Ex.: Lucas Oliveira" />
      <button class="button button-primary" onclick="addOwner()"> +
    </div>
  </div>
  <div>
    Adicionar</button>
  </div>
</div>

```

```

<div class="table-wrap" style="max-width: 500px;">
  <table>
    <thead>
      <tr>
        <th>Vendedor</th>
        <th style="width: 80px; text-align: right;">Ações</th>
      </tr>
    </thead>
    <tbody id="ownersTableBody"></tbody>
  </table>
</div>
</section>
</section>
</main>
</div>

```

```

<!-- MODAL: CREATE / EDIT DEAL -->

```

```

<div class="modal-backdrop" id="dealModal">
  <div class="modal">
    <div class="modal-header">
      <div>
        <h2 id="dealModalTitle">Novo negócio</h2>
        <p id="dealModalSubtitle">Registre uma oportunidade para acompanhar pelo
funil.</p>
      </div>
      <button class="close-button" onclick="closeModal('dealModal')">×</button>
    </div>

    <div class="modal-body">
      <form id="dealForm" onsubmit="saveDeal(event)">
        <input type="hidden" id="editingDealId" />

        <div class="form-grid">
          <div class="field">
            <label>Nome da empresa *</label>
            <input id="dealCompany" required placeholder="Ex.: Empresa ABC Ltda." />
          </div>

          <div class="field">
            <label>Nome do contato</label>
            <input id="dealContact" placeholder="Ex.: Maria Silva" />
          </div>

          <div class="field">
            <label>Telefone / WhatsApp</label>
            <input id="dealPhone" placeholder="Ex.: 41 99999-9999" />
          </div>

          <div class="field">
            <label>E-mail</label>
            <input id="dealEmail" type="email" placeholder="contato@empresa.com" />
          </div>

          <div class="field">
            <label>Valor estimado</label>
            <input id="dealValue" type="number" min="0" step="0.01" placeholder="Deixe em
branco se ainda não souber" />
          </div>

          <div class="field">
            <label>Probabilidade</label>
            <select id="dealProbability">
              <option value="10">10%</option>
              <option value="25">25%</option>
              <option value="40">40%</option>
            </select>
          </div>
        </div>
      </form>
    </div>
  </div>
</div>

```

```
<option value="60" selected>60%</option>
<option value="80">80%</option>
<option value="100">100%</option>
</select>
</div>
```

```
<div class="field">
  <label>Etapa</label>
  <select id="dealStage"></select>
</div>
```

```
<div class="field">
  <label>Responsável</label>
  <select id="dealOwner"></select>
</div>
```

```
<div class="field">
  <label>Previsão de fechamento</label>
  <input id="dealCloseDate" type="date" min="1960-01-01" max="2090-12-31" />
</div>
```

```
<div class="field">
  <label>Origem do lead</label>
  <select id="dealSource">
    <option value="">Não informada</option>
    <option>Site</option>
    <option>Indicação</option>
    <option>WhatsApp</option>
    <option>Prospecção ativa</option>
    <option>Instagram</option>
    <option>Evento</option>
  </select>
</div>
```

```
<div class="field full">
  <label>Tags (separadas por vírgula)</label>
  <input id="dealTags" placeholder="Ex.: prioridade, inbound, indústria" />
</div>
```

```
<div class="field full">
  <label>Observação</label>
  <textarea id="dealNote" placeholder="Contexto importante sobre a
oportunidade..."></textarea>
</div>
</div>
```

```
<div class="modal-actions">
```

```

        <button type="button" class="button button-secondary"
onclick="closeModal('dealModal')">Cancelar</button>
        <button class="button button-primary" type="submit">Salvar negócio</button>
    </div>
</form>
</div>
</div>
</div>

```

```

<!-- MODAL: ACTIVITY -->

```

```

<div class="modal-backdrop" id="activityModal">
  <div class="modal" style="max-width:700px">
    <div class="modal-header">
      <div>
        <h2>Nova atividade</h2>
        <p>A atividade será registrada na timeline e atualizada em todo o CRM.</p>
      </div>
      <button class="close-button" onclick="closeModal('activityModal')">×</button>
    </div>

```

```

    <div class="modal-body">
      <form onsubmit="createActivity(event)">
        <div class="form-grid">
          <div class="field">
            <label>Título *</label>
            <input id="activityTitle" required placeholder="Ex.: Ligar para o decisor" />
          </div>

```

```

          <div class="field">
            <label>Tipo *</label>
            <select id="activityType">
              <option>Ligação</option>
              <option>WhatsApp</option>
              <option>Reunião</option>
              <option>Visita</option>
              <option>Proposta</option>
              <option>Follow-up</option>
              <option>Tarefa</option>
            </select>
          </div>

```

```

          <div class="field">
            <label>Negócio relacionado *</label>
            <select id="activityDeal" required></select>
          </div>

```

```

          <div class="field">
            <label>Data e hora *</label>

```



```

        <input id="activityDate" required type="datetime-local" />
    </div>

    <div class="field">
        <label>Prioridade</label>
        <select id="activityPriority">
            <option>Baixa</option>
            <option selected>Média</option>
            <option>Alta</option>
        </select>
    </div>

    <div class="field">
        <label>Responsável</label>
        <select id="activityOwner"></select>
    </div>

    <div class="field full">
        <label>Descrição</label>
        <textarea id="activityDescription" placeholder="Descreva o próximo
passo..."></textarea>
    </div>

    <div class="modal-actions">
        <button type="button" class="button button-secondary"
onclick="closeModal('activityModal')>Cancelar</button>
        <button class="button button-primary" type="submit">Criar atividade</button>
    </div>
</form>
</div>
</div>
</div>

<!-- MODAL: DEAL DETAIL -->
<div class="modal-backdrop" id="detailModal">
    <div class="modal">
        <div class="modal-header" id="detailHeader"></div>

        <div class="modal-content">
            <aside class="deal-details" id="dealDetails"></aside>
            <section class="timeline" id="dealTimeline"></section>
        </div>
    </div>
</div>

<!-- MODAL: TRANSFER -->
<div class="modal-backdrop" id="transferModal">

```

```

<div class="modal small-modal">
  <div class="modal-header">
    <div>
      <h2>Transferir negócio</h2>
      <p>O novo responsável será registrado na timeline.</p>
    </div>
    <button class="close-button" onclick="closeModal('transferModal')">×</button>
  </div>

  <div class="modal-body">
    <input type="hidden" id="transferDealId" />

    <div class="field">
      <label>Novo responsável</label>
      <select id="transferOwner"></select>
    </div>

    <div class="modal-actions">
      <button class="button button-secondary"
onclick="closeModal('transferModal')">Cancelar</button>
      <button class="button button-primary" onclick="transferDeal()">Transferir</button>
    </div>
  </div>
</div>

<!-- MODAL: LOSS REASON -->
<div class="modal-backdrop" id="lossModal">
  <div class="modal small-modal">
    <div class="modal-header">
      <div>
        <h2>Marcar negócio</h2>
        <p>Informe se a oportunidade foi perdida ou representa churn.</p>
      </div>
      <button class="close-button" onclick="closeModal('lossModal')">×</button>
    </div>

    <div class="modal-body">
      <input type="hidden" id="lossDealId" />

      <div class="field">
        <label>Status</label>
        <select id="lossStatus">
          <option value="lost">Perdido</option>
          <option value="churn">Churn</option>
        </select>
      </div>
    </div>
  </div>
</div>

```

```

<div class="field" style="margin-top:14px">
  <label>Motivo</label>
  <select id="lossReason">
    <option>Preço</option>
    <option>Sem retorno do cliente</option>
    <option>Concorrência</option>
    <option>Sem fit</option>
    <option>Timing inadequado</option>
    <option>Outro</option>
  </select>
</div>

<div class="modal-actions">
  <button class="button button-secondary"
onclick="closeModal('lossModal')">Cancelar</button>
  <button class="button button-danger" onclick="markDealLost()">Confirmar</button>
</div>
</div>
</div>
</div>

<!-- MODAL: STAGE -->
<div class="modal-backdrop" id="stageModal">
  <div class="modal small-modal">
    <div class="modal-header">
      <div>
        <h2 id="stageModalTitle">Nova etapa</h2>
        <p>Defina o nome da etapa no funil comercial.</p>
      </div>
      <button class="close-button" onclick="closeModal('stageModal')">×</button>
    </div>

    <div class="modal-body">
      <input type="hidden" id="stageEditIndex" />

      <div class="field">
        <label>Nome da etapa *</label>
        <input id="stageName" placeholder="Ex.: Diagnóstico técnico" />
      </div>

      <div class="field" style="margin-top:14px">
        <label>Probabilidade sugerida</label>
        <select id="stageProbability">
          <option value="10">10%</option>
          <option value="20">20%</option>
          <option value="30">30%</option>
          <option value="40">40%</option>
          <option value="50">50%</option>
        </select>
      </div>
    </div>
  </div>
</div>

```

```

        <option value="60">60%</option>
        <option value="70">70%</option>
        <option value="80">80%</option>
        <option value="90">90%</option>
        <option value="100">100%</option>
    </select>
</div>

<div class="modal-actions">
    <button class="button button-secondary"
onclick="closeModal('stageModal')">Cancelar</button>
    <button class="button button-primary" onclick="saveStage()">Salvar etapa</button>
</div>
</div>
</div>
</div>

<script>
    // Data atualizada segura de acordo com o dia de acesso comercial (Sem distorção de
fuso horário)
    const CURRENT_DATE = new Date("2026-07-24T12:00:00");

    const DEFAULT_STAGES = [
        { name: "Novos leads para tentativa de contato", probability: 10 },
        { name: "Contato realizado", probability: 20 },
        { name: "Contato com o decisor", probability: 35 },
        { name: "Reunião", probability: 50 },
        { name: "Proposta enviada", probability: 65 },
        { name: "FUP", probability: 75 },
        { name: "FUP de fechamento", probability: 85 },
        { name: "Proposta aceita", probability: 100 }
    ];

    const defaultDeals = [
        {
            id: 1,
            company: "Grupo Horizonte",
            contact: "Mariana Souza",
            phone: "41 99999-1111",
            email: "mariana@grupohorizonte.com.br",
            value: 12000,
            probability: 10,
            stage: "Novos leads para tentativa de contato",
            owner: "Victor",
            closeDate: "2026-07-30",
            source: "Site",
            tags: "inbound, prioridade",
            status: "open",

```

```
    createdAt: "2026-07-24T08:30:00",
    timeline: [
      { title: "Negócio criado", description: "Oportunidade incluída por Victor.", date:
"2026-07-24T08:30:00" }
    ]
  },
  {
    id: 2,
    company: "Restaurante Sabor & Arte",
    contact: "Carlos Mendes",
    phone: "41 98888-2222",
    email: "",
    value: null,
    probability: 20,
    stage: "Contato realizado",
    owner: "Gustavo",
    closeDate: "",
    source: "WhatsApp",
    tags: "prospecção",
    status: "open",
    createdAt: "2026-07-20T10:20:00",
    timeline: [
      { title: "WhatsApp enviado", description: "Apresentação comercial enviada ao
contato.", date: "2026-07-20T10:20:00" }
    ]
  },
  {
    id: 3,
    company: "Clínica Vida Mais",
    contact: "Fernanda Lima",
    phone: "41 97777-3333",
    email: "fernanda@vidamais.com.br",
    value: 18000,
    probability: 50,
    stage: "Reunião",
    owner: "Victor",
    closeDate: "2026-07-25",
    source: "Indicação",
    tags: "saúde, decisor",
    status: "open",
    createdAt: "2026-07-17T09:00:00",
    timeline: [
      { title: "Reunião realizada", description: "Necessidade identificada: CRM para equipe
de 12 vendedores.", date: "2026-07-22T16:00:00" }
    ]
  },
  {
    id: 4,
```

```

    company: "Construtora Alpha",
    contact: "Rafael Costa",
    phone: "41 96666-4444",
    email: "rafael@alpha.com.br",
    value: 25000,
    probability: 65,
    stage: "Proposta enviada",
    owner: "Ana",
    closeDate: "2026-07-26",
    source: "Prospecção ativa",
    tags: "construção",
    status: "open",
    createdAt: "2026-07-15T15:20:00",
    timeline: [
      { title: "Proposta enviada", description: "Proposta comercial enviada por e-mail e
WhatsApp.", date: "2026-07-22T10:00:00" }
    ]
  },
  {
    id: 5,
    company: "Logística Paraná",
    contact: "João Ribeiro",
    phone: "41 95555-5555",
    email: "joao@logisticaparana.com.br",
    value: 30000,
    probability: 85,
    stage: "FUP de fechamento",
    owner: "Victor",
    closeDate: "2026-07-24",
    source: "Evento",
    tags: "alta prioridade",
    status: "open",
    createdAt: "2026-07-10T11:10:00",
    timeline: [
      { title: "Negociação em fechamento", description: "Cliente solicitou ajuste de
condições comerciais.", date: "2026-07-22T14:00:00" }
    ]
  }
];

const defaultActivities = [
  {
    id: 1,
    title: "Ligar para o decisor",
    type: "Ligação",
    dealId: 3,
    owner: "Victor",
    date: "2026-07-24T14:00",
  }
];

```

```

    priority: "Alta",
    description: "Validar participantes da reunião de diagnóstico.",
    status: "Agendada"
  },
  {
    id: 2,
    title: "Enviar proposta atualizada",
    type: "Proposta",
    dealId: 4,
    owner: "Ana",
    date: "2026-07-23T10:00",
    priority: "Alta",
    description: "Inserir condições comerciais revisadas.",
    status: "Agendada"
  },
  {
    id: 3,
    title: "Follow-up comercial",
    type: "WhatsApp",
    dealId: 5,
    owner: "Victor",
    date: "2026-07-24T16:30",
    priority: "Média",
    description: "Confirmar retorno da proposta revisada.",
    status: "Agendada"
  },
  {
    id: 4,
    title: "Atualizar cadastro",
    type: "Tarefa",
    dealId: 1,
    owner: "Victor",
    date: "2026-07-24T09:30",
    priority: "Baixa",
    description: "",
    status: "Concluída"
  }
];

```

```

let stages = DEFAULT_STAGES;
let deals = defaultDeals;
let activities = defaultActivities;
let forecastGoal = 80000;
let owners = ["Victor", "Gustavo", "Ana"];

```

```

// Tratamento robusto do LocalStorage
try {
  const storedOwners = localStorage.getItem("salesmates_owners");

```

```

    if (storedOwners) {
      const parsed = JSON.parse(storedOwners);
      if (Array.isArray(parsed) && parsed.length > 0) owners = parsed.filter(Boolean);
    }
  } catch (e) { console.error(e); }

  try {
    const storedStages = localStorage.getItem("salesmates_stages_v3");
    if (storedStages) {
      const parsed = JSON.parse(storedStages);
      if (Array.isArray(parsed) && parsed.length > 0) {
        stages = parsed.map((item, i) => {
          if (typeof item === 'string') {
            return { name: item, probability: DEFAULT_STAGES[i] ?
DEFAULT_STAGES[i].probability : 50 };
          } else if (item && typeof item === 'object') {
            return {
              name: item.name || `Etapă ${i + 1}`,
              probability: typeof item.probability !== 'undefined' ? Number(item.probability) : 50
            };
          }
        });
        return null;
      }
    }
  } catch (e) { console.error(e); stages = DEFAULT_STAGES; }

  try {
    const storedDeals = localStorage.getItem("salesmates_deals_v3");
    if (storedDeals) {
      const parsed = JSON.parse(storedDeals);
      if (Array.isArray(parsed)) deals = parsed.filter(Boolean);
    }
  } catch (e) { console.error(e); deals = defaultDeals; }

  try {
    const storedActivities = localStorage.getItem("salesmates_activities_v3");
    if (storedActivities) {
      const parsed = JSON.parse(storedActivities);
      if (Array.isArray(parsed)) activities = parsed.filter(Boolean);
    }
  } catch (e) { console.error(e); activities = defaultActivities; }

  try {
    const storedGoal = localStorage.getItem("salesmates_goal_v3");
    if (storedGoal) {
      const parsedGoal = Number(storedGoal);
      if (!isNaN(parsedGoal) && parsedGoal >= 0) forecastGoal = parsedGoal;
    }
  }

```



```

    }
  } catch (e) { console.error(e); }

let selectedDate = new Date("2026-07-24T12:00:00");
let calendarCursor = new Date(selectedDate.getFullYear(), selectedDate.getMonth(), 1);
let agendaView = "day";
let pipelineView = "kanban";
let currentDetailDealId = null;

function stageNames() { return stages.map((s) => s ? s.name : "").filter(Boolean); }
function getStageData(name) { return stages.find((s) => s && s.name === name) ||
stages[0] || DEFAULT_STAGES[0]; }

function migrateOldDeals() {
  deals.forEach((deal) => {
    if (!deal) return;
    if (!stageNames().includes(deal.stage)) {
      deal.stage = stages[0] ? stages[0].name : DEFAULT_STAGES[0].name;
      deal.probability = stages[0] ? stages[0].probability : DEFAULT_STAGES[0].probability;
    }
  });
}

function saveData() {
  localStorage.setItem("salesmates_stages_v3", JSON.stringify(stages));
  localStorage.setItem("salesmates_deals_v3", JSON.stringify(deals));
  localStorage.setItem("salesmates_activities_v3", JSON.stringify(activities));
  localStorage.setItem("salesmates_goal_v3", String(forecastGoal));
  localStorage.setItem("salesmates_owners", JSON.stringify(owners));
}

// EXPORTAR BACKUP JSON
function exportBackup() {
  const backupData = {
    stages,
    deals,
    activities,
    forecastGoal,
    owners
  };
  const blob = new Blob([JSON.stringify(backupData, null, 2)], { type: "application/json" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = `salesmates_backup_${dateKey(new Date())}.json`;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
}

```

```

    URL.revokeObjectURL(url);
}

function triggerImportBackup() {
    document.getElementById("backupFileInput").click();
}

// IMPORTAR BACKUP JSON
function importBackup(event) {
    const file = event.target.files[0];
    if (!file) return;

    const reader = new FileReader();
    reader.onload = function(e) {
        try {
            const imported = JSON.parse(e.target.result);
            if (imported.stages) stages = imported.stages;
            if (imported.deals) deals = imported.deals;
            if (imported.activities) activities = imported.activities;
            if (imported.forecastGoal) forecastGoal = Number(imported.forecastGoal);
            if (imported.owners) owners = imported.owners;

            saveData();
            alert("Backup restaurado com sucesso!");
            location.reload();
        } catch (err) {
            alert("Erro ao ler o arquivo de backup. Verifique se é um arquivo JSON válido do SalesMates.");
        }
    };
    reader.readAsText(file);
}

function escapeHTML(value) {
    return String(value || "")
        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;")
        .replace(/'/g, "&#039;");
}

function currency(value) {
    if (value === null || value === undefined || value === "" || Number(value) <= 0) {
        return "R$ a definir";
    }
    return new Intl.NumberFormat("pt-BR", {
        style: "currency", currency: "BRL", maximumFractionDigits: 0
    }).format(value);
}

```

```

    }).format(Number(value));
}

function currencyNumber(value) {
    return new Intl.NumberFormat("pt-BR", {
        style: "currency", currency: "BRL", maximumFractionDigits: 0
    }).format(Number(value || 0));
}

function formatDate(date, options = {}) {
    if (!date) return "Não informado";
    const d = new Date(date + "T12:00:00"); // Sanitização de fuso horário local
    if (isNaN(d.getTime())) return "Não informado";
    return d.toLocaleDateString("pt-BR", {
        day: "2-digit", month: "2-digit", year: "numeric", ...options
    });
}

function formatDateTime(date) {
    if (!date) return "Não informado";
    const d = new Date(date);
    if (isNaN(d.getTime())) return "Não informado";
    return d.toLocaleString("pt-BR", {
        day: "2-digit", month: "2-digit", year: "numeric", hour: "2-digit", minute: "2-digit"
    });
}

function dateKey(date) {
    if (!date) return "";
    const d = new Date(date);
    if (isNaN(d.getTime())) return "";
    return d.toISOString().split("T")[0];
}

function getDeal(id) { return deals.find((d) => d && Number(d.id) === Number(id)); }
function getDealActivities(dealId) { return activities.filter((a) => a && Number(a.dealId) === Number(dealId)); }

function getActivityStatus(activity) {
    if (!activity) return "Cancelada";
    if (activity.status === "Concluída") return "Concluída";
    if (activity.status === "Cancelada") return "Cancelada";
    return new Date(activity.date) < CURRENT_DATE ? "Atrasada" : "Agendada";
}

function getNextActivity(dealId) {
    return getDealActivities(dealId)
        .filter((a) => a && ![ "Concluída", "Cancelada" ].includes(getActivityStatus(a)))

```

```

        .sort((a, b) => new Date(a.date) - new Date(b.date))[0];
    }

    function addTimelineEvent(deal, title, description) {
        if (!deal) return;
        if (!deal.timeline) deal.timeline = [];
        deal.timeline.unshift({ title, description, date: new Date().toISOString() });
    }

    function activityIcon(type) {
        const icons = { "Ligação": "☎", "WhatsApp": "☎", "Reunião": "🗓", "Visita": "📍", "Proposta": "📋", "Follow-up": "🔄", "Tarefa": "✓" };
        return icons[type] || "✓";
    }

    function badgeForStatus(status) {
        const classes = { "Concluída": "badge-green", "Atrasada": "badge-red", "Agendada": "badge-blue", "Cancelada": "badge-gray", "Ganho": "badge-green", "Perdido": "badge-red", "Churn": "badge-yellow", "Ativo": "badge-blue" };
        return `<span class="badge ${classes[status]} || "badge-gray">${escapeHTML(status)}</span>`;
    }

    function dealStatusLabel(status) {
        const labels = { open: "Ativo", won: "Ganho", lost: "Perdido", churn: "Churn", archived: "Arquivado" };
        return labels[status] || status;
    }

    function dealValueHTML(deal) {
        return (deal && Number(deal.value) > 0)
            ? `<div class="deal-value">${currencyNumber(deal.value)}</div>`
            : `<div class="deal-value" style="color:#6b7280;font-weight:500;">R$ a definir</div>`;
    }

    function fillOwnerOptions() {
        const options = owners.map((o) => `<option value="${escapeHTML(o)}">${escapeHTML(o)}</option>`).join("");
        ["pipelineOwner", "activityOwnerFilter", "forecastOwnerFilter"].forEach((id) => {
            const select = document.getElementById(id);
            if (!select) return;
            const current = select.value;
            const firstOption = select.options[0]?.outerHTML || "";
            select.innerHTML = firstOption + options;
            select.value = current;
        });
        ["dealOwner", "activityOwner", "transferOwner"].forEach((id) => {
            const select = document.getElementById(id);

```

```

    if (select) select.innerHTML = options;
  });
}

function renderOwnersList() {
  const tbody = document.getElementById("ownersTableBody");
  if (!tbody) return;
  tbody.innerHTML = owners.map((o) => `
    <tr>
      <td>
        <div style="display:flex; align-items:center; gap:8px;">
          <span class="small-avatar">${o ? o.charAt(0) : "V"}</span>
          <strong>${escapeHTML(o)}</strong>
        </div>
      </td>
      <td style="text-align: right;">
        <button class="icon-button danger" title="Remover Vendedor"
onclick="removeOwner('${escapeHTML(o)}')">x</button>
      </td>
    </tr>
  `).join("");
}

function addOwner() {
  const input = document.getElementById("newOwnerName");
  const name = input?.value.trim();
  if (!name) return;
  if (owners.includes(name)) { alert("Este vendedor já existe."); return; }
  owners.push(name);
  saveData();
  input.value = "";
  refreshAll();
  renderOwnersList();
}

function removeOwner(name) {
  if (owners.length <= 1) { alert("Mantenha ao menos um vendedor."); return; }
  if (confirm(`Remover "${name}"?`)) {
    owners = owners.filter((o) => o !== name);
    saveData();
    refreshAll();
    renderOwnersList();
  }
}

function refreshAll() {
  fillOwnerOptions();
  renderDashboard();
}

```

```

renderKanban();
renderActivities();
renderForecast();
renderTeam();
renderReports();
renderStageEditor();
}

function showView(viewId) {
  document.querySelectorAll(".view").forEach((v) => v.classList.remove("active"));
  document.querySelectorAll(".nav-button").forEach((b) => b.classList.remove("active"));
  const targetView = document.getElementById(viewId);
  if (targetView) targetView.classList.add("active");
  const targetNav = document.querySelector(`[data-view="${viewId}"]`);
  if (targetNav) targetNav.classList.add("active");

  if (viewId === "pipeline") renderKanban();
  if (viewId === "activities") { renderCalendar(); renderActivities(); }
  if (viewId === "forecast") renderForecast();
  if (viewId === "team") renderTeam();
  if (viewId === "reports") renderReports();
  if (viewId === "settings") {
    const goalInput = document.getElementById("goalInput");
    if (goalInput) goalInput.value = forecastGoal;
    renderOwnersList();
  }
}

document.querySelectorAll(".nav-button").forEach((b) => {
  b.addEventListener("click", () => showView(b.dataset.view));
});

function renderConversionRows(containerId) {
  const container = document.getElementById(containerId);
  if (!container) return;
  const openDeals = deals.filter((d) => d && d.status === "open");

  container.innerHTML = stages.map((stage, index) => {
    if (!stage) return "";
    const currentCount = openDeals.filter((d) => d && d.stage === stage.name).length;
    const previousCount = index === 0 ? currentCount : openDeals.filter((d) => {
      if (!d) return false;
      const dIndex = stageNames().indexOf(d.stage);
      return dIndex >= 0 && dIndex <= index - 1;
    }).length;

    const reachedCount = openDeals.filter((d) => {
      if (!d) return false;

```

```

        const dIndex = stageNames().indexOf(d.stage);
        return dIndex >= index;
    }).length;

    const conversion = index === 0 ? 100 : (previousCount > 0 ?
Math.round((reachedCount / previousCount) * 100) : 0);
    const totalValue = openDeals.filter((d) => d && d.stage === stage.name).reduce((sum,
d) => sum + Number(d.value || 0), 0);

    return `
    <div class="conversion-row">
    <div class="conversion-top">
    <span>${escapeHTML(stage.name)}</span>
    <strong>${conversion}%</strong>
    </div>
    <div class="progress-track">
    <div class="progress-bar" style="width:${conversion}%;background:${index % 2 ?
"var(--primary)" : "#10b981"}"></div>
    </div>
    <div class="conversion-meta">
    ${currentCount} negócio(s) · ${currencyNumber(totalValue)}
    ${index === 0 ? " · Base do funil" : " · Conversão da etapa anterior"}
    </div>
    </div>
    `;
    }).join("");
}

function renderDashboard() {
    const currentDay = dateKey(CURRENT_DATE);
    const activeActivities = activities.filter(Boolean);

    const completedToday = activeActivities.filter((a) =>
    getActivityStatus(a) === "Concluída" && dateKey(a.date) === currentDay
    ).length;

    const pending = activeActivities.filter((a) => getActivityStatus(a) === "Agendada").length;
    const overdue = activeActivities.filter((a) => getActivityStatus(a) === "Atrasada").length;

    const weightedForecast = deals.filter((d) => d && d.status === "open").reduce((total, d)
=> {
    return total + Number(d.value || 0) * (Number(d.probability || 0) / 100);
    }, 0);

    const todayCompletedEl = document.getElementById("todayCompleted");
    if (todayCompletedEl) todayCompletedEl.textContent = completedToday;

    const pendingActivitiesEl = document.getElementById("pendingActivities");

```

```

if (pendingActivitiesEl) pendingActivitiesEl.textContent = pending;

const overdueActivitiesEl = document.getElementById("overdueActivities");
if (overdueActivitiesEl) overdueActivitiesEl.textContent = overdue;

const weeklyForecastEl = document.getElementById("weeklyForecast");
if (weeklyForecastEl) weeklyForecastEl.textContent =
currencyNumber(weightedForecast);

renderConversionRows("conversionDashboard");

const nextActivities = [...activeActivities]
  .filter((a) => ![ "Concluída", "Cancelada" ].includes(getActivityStatus(a)))
  .sort((a, b) => new Date(a.date) - new Date(b.date))
  .slice(0, 4);

const dashboardActivitiesEl = document.getElementById("dashboardActivities");
if (dashboardActivitiesEl) {
  dashboardActivitiesEl.innerHTML = nextActivities.length
    ? nextActivities.map((a) => {
      const d = getDeal(a.dealId);
      const status = getActivityStatus(a);
      return `
        <div class="activity-item">
          <div class="activity-icon">${activityIcon(a.type)}</div>
          <div class="activity-info">
            <strong>${escapeHTML(a.title)}</strong>
            <small>${escapeHTML(d?.company || "Sem negócio")} ·
${formatDateTime(a.date)}</small>
          </div>
          ${badgeForStatus(status)}
        </div>
      `;
    }).join("")
    : `<div class="empty">Nenhuma atividade pendente.</div>`;
}

function getFilteredPipelineDeals() {
  const search = document.getElementById("pipelineSearch")?.value.trim().toLowerCase()
  || "";
  const owner = document.getElementById("pipelineOwner")?.value || "";
  const activityFilter = document.getElementById("pipelineActivity")?.value || "";
  const status = document.getElementById("pipelineStatus")?.value || "open";
  const lossReason = document.getElementById("pipelineLossReason")?.value || "";

  return deals.filter((deal) => {
    if (!deal) return false;

```



```

const matchesStatus = status === "all" || deal.status === status;

const fields = [
  deal.company, deal.contact, deal.phone, deal.email, deal.tags, deal.lossReason
].join(" ").toLowerCase();

const matchesSearch = !search || fields.includes(search);
const matchesOwner = !owner || deal.owner === owner;
const matchesLossReason = !lossReason || deal.lossReason === lossReason;

const next = getNextActivity(deal.id);
const nextStatus = next ? getActivityStatus(next) : null;

const matchesActivity =
  !activityFilter ||
  (activityFilter === "overdue" && nextStatus === "Atrasada") ||
  (activityFilter === "none" && !next);

return matchesStatus && matchesSearch && matchesOwner && matchesLossReason
&& matchesActivity;
});
}

function renderKanban() {
  const board = document.getElementById("kanbanBoard");
  const list = document.getElementById("pipelineList");
  if (!board || !list) return;

  const filteredDeals = getFilteredPipelineDeals();

  board.style.display = pipelineView === "kanban" ? "flex" : "none";
  list.style.display = pipelineView === "list" ? "block" : "none";

  if (pipelineView === "list") {
    list.innerHTML = `
      <table>
        <thead>
          <tr>
            <th>Negócio</th>
            <th>Status</th>
            <th>Etapa</th>
            <th>Responsável</th>
            <th>Valor</th>
            <th>Próxima ação</th>
          </tr>
        </thead>
        <tbody>
          ${filteredDeals.length ? filteredDeals.map((deal) => {

```

```

    if (!deal) return "";
    const next = getNextActivity(deal.id);
    const nextStatus = next ? getActivityStatus(next) : "";
    return `
      <tr onclick="openDealDetail(${deal.id})" style="cursor:pointer">
        <td>
          <strong>${escapeHTML(deal.company)}</strong><br>
          <small style="color:var(--muted)">${escapeHTML(deal.contact || "Sem
contato")}</small>
        </td>
        <td>${badgeForStatus(dealStatusLabel(deal.status))}</td>
        <td>${escapeHTML(deal.stage)}</td>
        <td>${escapeHTML(deal.owner)}</td>
        <td>${currency(deal.value)}</td>
        <td style="color:${nextStatus === "Atrasada" ? "var(--danger)" : "inherit"}">
          ${next ? `${nextStatus === "Atrasada" ? "Vencida: " :
""}${escapeHTML(next.title)}` : "Sem atividade"}
        </td>
      </tr>
    `;
    }).join("") : `<tr><td colspan="6"><div class="empty">Nenhum negócio
encontrado.</div></td></tr>`;
  </tbody>
</table>
`;
  return;
}

```

```

const activeStatus = document.getElementById("pipelineStatus")?.value || "open";

```

```

board.innerHTML = stages.map((stage) => {
  if (!stage) return "";
  const stageDeals = filteredDeals.filter((deal) => deal && deal.stage === stage.name);
  const stageTotal = stageDeals.reduce((sum, deal) => sum + Number(deal.value || 0),
0);

```

```

  return `
    <div
      class="kanban-column"
      ondragover="event.preventDefault()"
      ondrop="dropDeal(event, '${escapeHTML(stage.name)}')">
      >
        <div class="column-header">
          <div>
            <span class="column-title"
title="${escapeHTML(stage.name)}">${escapeHTML(stage.name)}</span>
            <span class="column-total">${currencyNumber(stageTotal)} ·
${stage.probability}%</span>

```

```

    </div>
    <span class="column-count">${stageDeals.length}</span>
  </div>

  ${stageDeals.map((deal) => {
    if (!deal) return "";
    const next = getNextActivity(deal.id);
    const nextStatus = next ? getActivityStatus(next) : null;

    let dotClass = "gray";
    if (nextStatus === "Atrasada") {
      dotClass = "red";
    } else if (next) {
      const isToday = dateKey(next.date) === dateKey(CURRENT_DATE);
      dotClass = isToday ? "yellow" : "green";
    }

    const tagsArray = deal.tags ? deal.tags.split(",").map(t => t.trim()).filter(Boolean) : [];

    return `
    <article
      class="deal-card"
      draggable="${activeStatus === "open" ? "true" : "false"}"
      ondragstart="dragDeal(event, ${deal.id})"
      onclick="openDealDetail(${deal.id})"
    >
      <div>
        <h3>${escapeHTML(deal.company)}</h3>
        <p style="margin-top: 2px;">${escapeHTML(deal.contact || "Sem contato")}</p>
      </div>

      ${dealValueHTML(deal)}

      ${tagsArray.length ? `
        <div class="deal-card-tags">
          ${tagsArray.map(tag => `<span
class="deal-card-tag">${escapeHTML(tag)}</span>`).join("")}
        </div>
      ` : ""}

      <div class="deal-card-footer">
        <div class="deal-card-owner">
          <span class="small-avatar" style="width: 18px; height: 18px; font-size: 8px;">
            ${deal.owner ? deal.owner.charAt(0) : "V"}
          </span>
          <span>${escapeHTML(deal.owner)}</span>
        </div>

```

```

        <span
            class="activity-dot ${dotClass}"
            title="${next ? `Próxima: ${next.title} (${formatDate(next.date)})` : 'Sem
atividades'}"
        ></span>
    </div>
</article>
`;
}).join("")

```

```

        ${!stageDeals.length ? `<div style="padding:16px
5px;color:var(--muted);font-size:11px;text-align:center">Nenhum negócio</div>` : ""}
        </div>
        `;
    }).join("");
}

```

```

function setPipelineView(view) {
    pipelineView = view;
    const kToggle = document.getElementById("kanbanToggle");
    if (kToggle) kToggle.classList.toggle("active", view === "kanban");
    const lToggle = document.getElementById("listToggle");
    if (lToggle) lToggle.classList.toggle("active", view === "list");
    renderKanban();
}

```

```

function dragDeal(event, dealId) { event.dataTransfer.setData("dealId", dealId); }

```

```

function dropDeal(event, stage) {
    const dealId = Number(event.dataTransfer.getData("dealId"));
    const deal = getDeal(dealId);
    if (!deal || deal.status !== "open" || deal.stage === stage) return;

```

```

    const oldStage = deal.stage;
    deal.stage = stage;
    deal.probability = getStageData(stage).probability;

```

```

    addTimelineEvent(deal, "Etapa alterada", `${oldStage} → ${stage}. Probabilidade para
    ${deal.probability}%`);
    saveData();
    refreshAll();
}

```

```

function updateDealStageDirectly(dealId, newStageName) {
    const deal = getDeal(dealId);
    if (!deal || deal.status !== "open" || deal.stage === newStageName) return;

    const oldStage = deal.stage;

```

```

deal.stage = newStageName;
deal.probability = getStageData(newStageName).probability;

addTimelineEvent(deal, "Etapa alterada (Chevron)", `${oldStage} →
${newStageName}.`);
saveData();
refreshAll();
openDealDetail(dealId);
}

function togglePipelineManager() {
  const pm = document.getElementById("pipelineManager");
  if (pm) pm.classList.toggle("open");
  renderStageEditor();
}

function renderStageEditor() {
  const container = document.getElementById("stageEditor");
  if (!container) return;

  container.innerHTML = stages.map((stage, index) => {
    if (!stage) return "";
    return `
      <div class="stage-editor-card">
        <strong title="${escapeHTML(stage.name)}">${escapeHTML(stage.name)}</strong>
        <small style="display:block;margin-bottom:9px;color:var(--muted);font-size:10px">
          Probabilidade sugerida: ${stage.probability}%
        </small>

        <div class="stage-editor-actions">
          <button class="icon-button" title="Esquerda" ${index === 0 ? "disabled" : ""}
onclick="moveStage(${index}, -1)">←</button>
          <button class="icon-button" title="Editar"
onclick="openStageModal(${index})">✎</button>
          <button class="icon-button" title="Direita" ${index === stages.length - 1 ? "disabled"
: ""} onclick="moveStage(${index}, 1)">→</button>
          <button class="icon-button danger" title="Excluir"
onclick="deleteStage(${index})">×</button>
        </div>
      </div>
      <button class="stage-add-between" title="Inserir Etapa"
onclick="openStageModal(null, ${index + 1})">+</button>
    `;
  }).join("");

  container.insertAdjacentHTML("afterbegin", `<button class="stage-add-between"
title="Inserir Inicial" onclick="openStageModal(null, 0)">+</button>`);
}

```

```

function openStageModal(index = null, insertAt = null) {
  const stageEditIndexInput = document.getElementById("stageEditIndex");
  if (!stageEditIndexInput) return;

  stageEditIndexInput.value = index === null ? `new:${insertAt}` : index;
  const titleEl = document.getElementById("stageModalTitle");
  if (titleEl) titleEl.textContent = index === null ? "Criar etapa" : "Editar etapa";

  const nameEl = document.getElementById("stageName");
  const probEl = document.getElementById("stageProbability");

  if (index === null) {
    if (nameEl) nameEl.value = "";
    if (probEl) probEl.value = "50";
  } else {
    if (nameEl) nameEl.value = stages[index].name;
    if (probEl) probEl.value = stages[index].probability;
  }
  const modalEl = document.getElementById("stageModal");
  if (modalEl) modalEl.classList.add("open");
}

function saveStage() {
  const reference = document.getElementById("stageEditIndex")?.value;
  const name = document.getElementById("stageName")?.value.trim();
  const probability = Number(document.getElementById("stageProbability")?.value || 50);

  if (!name) { alert("Informe um nome."); return; }

  if (reference && reference.startsWith("new:")) {
    const insertAt = Number(reference.split(":")[1]);
    stages.splice(insertAt, 0, { name, probability });
  } else if (reference !== undefined) {
    const index = Number(reference);
    const oldName = stages[index].name;
    stages[index] = { name, probability };

    deals.forEach((d) => {
      if (d && d.stage === oldName) {
        d.stage = name;
        d.probability = probability;
        addTimelineEvent(d, "Etapa renomeada", `${oldName} → ${name}.`);
      }
    });
  }

  saveData();
}

```

```

    closeModal("stageModal");
    refreshAll();
}

function moveStage(index, direction) {
    const newIndex = index + direction;
    if (newIndex < 0 || newIndex >= stages.length) return;
    [stages[index], stages[newIndex]] = [stages[newIndex], stages[index]];
    saveData();
    refreshAll();
}

function deleteStage(index) {
    if (stages.length <= 2) { alert("Mantenha ao menos duas etapas."); return; }
    const stage = stages[index];
    const destination = stages[index - 1] || stages[index + 1];

    if (!confirm(`Excluir a etapa "${stage.name}"? Os negócios serão transferidos para
"${destination.name}"`)) return;

    deals.forEach((d) => {
        if (d && d.stage === stage.name) {
            d.stage = destination.name;
            d.probability = destination.probability;
            addTimelineEvent(d, "Etapas removida", `${stage.name} excluída. Movido para
${destination.name}.`);
        }
    });

    stages.splice(index, 1);
    saveData();
    refreshAll();
}

function renderCalendar() {
    const title = document.getElementById("calendarTitle");
    const container = document.getElementById("calendarDays");
    if (!title || !container) return;

    title.textContent = calendarCursor.toLocaleDateString("pt-BR", { month: "long", year:
"numeric" });

    const year = calendarCursor.getFullYear();
    const month = calendarCursor.getMonth();
    const firstDay = new Date(year, month, 1).getDay();
    const lastDate = new Date(year, month + 1, 0).getDate();

    let html = "";

```

```

for (let i = 0; i < firstDay; i++) {
  html += `<span class="calendar-day empty-day"></span>`;
}

for (let day = 1; day <= lastDate; day++) {
  const thisDate = new Date(year, month, day, 12, 0, 0);
  const key = dateKey(thisDate);
  const hasActivity = activities.some((a) => a && dateKey(a.date) === key);
  const selected = dateKey(selectedDate) === key;
  const today = dateKey(CURRENT_DATE) === key;

  html += `
    <button
      class="calendar-day ${selected ? "selected" : ""} ${today ? "today" : ""} ${hasActivity
? "has-activity" : ""}"
      onclick="selectCalendarDate('${key}')"
    >
      ${day}
    </button>
  `;
}
container.innerHTML = html;
}

function changeCalendarMonth(direction) {
  calendarCursor = new Date(calendarCursor.getFullYear(), calendarCursor.getMonth() +
direction, 1);
  renderCalendar();
}

function selectCalendarDate(key) {
  selectedDate = new Date(key + "T12:00:00");
  calendarCursor = new Date(selectedDate.getFullYear(), selectedDate.getMonth(), 1);
  renderCalendar();
  renderActivities();
}

function goToday() {
  selectedDate = new Date(CURRENT_DATE);
  calendarCursor = new Date(CURRENT_DATE.getFullYear(),
CURRENT_DATE.getMonth(), 1);
  renderCalendar();
  renderActivities();
}

function setAgendaView(view) {
  agendaView = view;
  ["day", "week", "month", "year", "list"].forEach((item) => {

```



```

    const el = document.getElementById(`agenda${item.charAt(0).toUpperCase() +
item.slice(1)}Toggle`);
    el?.classList.toggle("active", item === view);
  });
  renderActivities();
}

function isInAgendaPeriod(date) {
  const activityDate = new Date(date);
  const selected = new Date(selectedDate);
  if (agendaView === "list") return true;

  if (agendaView === "day") {
    return dateKey(activityDate) === dateKey(selected);
  }
  if (agendaView === "month") {
    return activityDate.getFullYear() === selected.getFullYear() && activityDate.getMonth()
=== selected.getMonth();
  }
  if (agendaView === "year") {
    return activityDate.getFullYear() === selected.getFullYear();
  }

  const firstDayOfWeek = new Date(selected);
  firstDayOfWeek.setDate(selected.getDate() - selected.getDay());
  firstDayOfWeek.setHours(0, 0, 0, 0);

  const lastDayOfWeek = new Date(firstDayOfWeek);
  lastDayOfWeek.setDate(firstDayOfWeek.getDate() + 7);

  return activityDate >= firstDayOfWeek && activityDate < lastDayOfWeek;
}

function getAgendaLabel() {
  if (agendaView === "list") return "Todas as atividades";
  if (agendaView === "year") return `Ano de ${selectedDate.getFullYear()}`;
  if (agendaView === "month") return selectedDate.toLocaleDateString("pt-BR", { month:
"long", year: "numeric" });
  if (agendaView === "week") {
    const first = new Date(selectedDate);
    first.setDate(selectedDate.getDate() - selectedDate.getDay());
    const last = new Date(first);
    last.setDate(first.getDate() + 6);
    return `${formatDate(first.toISOString().split("T")[0])} —
${formatDate(last.toISOString().split("T")[0])}`;
  }
  return selectedDate.toLocaleDateString("pt-BR", { weekday: "long", day: "2-digit", month:
"long", year: "numeric" });
}

```

```

}

function renderActivities() {
  const table = document.getElementById("activitiesTable");
  if (!table) return;

  const owner = document.getElementById("activityOwnerFilter")?.value || "";
  const type = document.getElementById("activityTypeFilter")?.value || "";
  const statusFilter = document.getElementById("activityStatusFilter")?.value || "";
  const search = document.getElementById("activitySearch")?.value.trim().toLowerCase()
  || "";

  const activeActivities = activities.filter(Boolean);

  const filtered = activeActivities
    .map((a) => ({ ...a, calculatedStatus: getActivityStatus(a) }))
    .filter((a) => {
      const d = getDeal(a.dealId);
      const matchesPeriod = isInAgendaPeriod(a.date);
      const matchesOwner = !owner || a.owner === owner;
      const matchesType = !type || a.type === type;
      const matchesStatus = !statusFilter || a.calculatedStatus === statusFilter;
      const matchesSearch = !search || a.title.toLowerCase().includes(search) ||
      (d?.company || "").toLowerCase().includes(search);

      return matchesPeriod && matchesOwner && matchesType && matchesStatus &&
      matchesSearch;
    })
    .sort((a, b) => new Date(a.date) - new Date(b.date));

  const labelEl = document.getElementById("agendaDateLabel");
  if (labelEl) labelEl.textContent = getAgendaLabel();

  const periodActivities = activeActivities.filter((a) => isInAgendaPeriod(a.date));
  const scheduled = activeActivities.filter((a) => getActivityStatus(a) ===
  "Agendada").length;
  const overdue = activeActivities.filter((a) => getActivityStatus(a) === "Atrasada").length;
  const completed = activeActivities.filter((a) => getActivityStatus(a) ===
  "Concluída").length;

  const schedEl = document.getElementById("agendaScheduled"); if (schedEl)
  schedEl.textContent = scheduled;
  const overEl = document.getElementById("agendaOverdue"); if (overEl)
  overEl.textContent = overdue;
  const compEl = document.getElementById("agendaCompleted"); if (compEl)
  compEl.textContent = completed;

```

```

    const pPending = periodActivities.filter((a) => getActivityStatus(a) ===
"Agendada").length;
    const pOverdue = periodActivities.filter((a) => getActivityStatus(a) ===
"Atrasada").length;
    const pCompleted = periodActivities.filter((a) => getActivityStatus(a) ===
"Concluída").length;
    const max = Math.max(pPending, pOverdue, pCompleted, 1);

    const chPend = document.getElementById("chartPending"); if (chPend)
chPend.textContent = pPending;
    const chOver = document.getElementById("chartOverdue"); if (chOver)
chOver.textContent = pOverdue;
    const chComp = document.getElementById("chartCompleted"); if (chComp)
chComp.textContent = pCompleted;

    const barPend = document.getElementById("chartPendingBar"); if (barPend)
barPend.style.width = `${(pPending / max) * 100}%`;
    const barOver = document.getElementById("chartOverdueBar"); if (barOver)
barOver.style.width = `${(pOverdue / max) * 100}%`;
    const barComp = document.getElementById("chartCompletedBar"); if (barComp)
barComp.style.width = `${(pCompleted / max) * 100}%`;

    table.innerHTML = filtered.length
    ? filtered.map((a) => {
        const d = getDeal(a.dealId);
        return `
            <tr>
            <td>
                <strong>${escapeHTML(a.title)}</strong><br>
                <small style="color:var(--muted)">${escapeHTML(a.priority)} prioridade</small>
            </td>
            <td>${escapeHTML(a.type)}</td>
            <td>
                <button class="link-button" onclick="openDealDetail(${a.dealId})">
                    ${escapeHTML(d?.company || "Sem vínculo")}
                </button>
            </td>
            <td>${escapeHTML(a.owner)}</td>
            <td>${formatDateTime(a.date)}</td>
            <td>${badgeForStatus(a.calculatedStatus)}</td>
            <td>
                ${a.calculatedStatus !== "Concluída" && a.calculatedStatus !== "Cancelada"
                ? `<button class="button button-primary button-small"
onclick="completeActivity(${a.id})">Concluir</button>`
                : ""}
            </td>
            </tr>
        `;
    })
    : `
    `;

```

```

    }).join("")
    : `<tr><td colspan="7"><div class="empty">Nenhuma atividade
    agendada.</div></td></tr>`;
  }

  function completeActivity(activityId) {
    const a = activities.find((item) => item.id === Number(activityId));
    if (!a) return;
    a.status = "Concluída";
    const d = getDeal(a.dealId);
    if (d) addTimelineEvent(d, `Atividade concluída: ${a.title}`, `${a.type} concluída por
    ${a.owner}.`);

    saveData();
    refreshAll();
    if (currentDetailDealId === a.dealId &&
    document.getElementById("detailModal")?.classList.contains("open")) {
      openDealDetail(currentDetailDealId);
    }
  }

  function renderForecast() {
    const owner = document.getElementById("forecastOwnerFilter")?.value || "";

    const openDeals = deals.filter((d) => d.status === "open" && (!owner || d.owner
    === owner));
    const dealsWithValue = openDeals.filter((d) => d.value > 0);
    const dealsWithoutValue = openDeals.filter((d) => d.value < 0);

    const gross = dealsWithValue.reduce((sum, d) => sum + Number(d.value), 0);
    const weighted = dealsWithValue.reduce((sum, d) => sum + Number(d.value) *
    (Number(d.probability) / 100), 0);

    const gap = Math.max(forecastGoal - weighted, 0);
    const goalProgress = Math.min((weighted / forecastGoal) * 100, 100);

    const fGoal = document.getElementById("forecastGoal"); if (fGoal) fGoal.textContent =
    currencyNumber(forecastGoal);
    const fGross = document.getElementById("grossForecast"); if (fGross)
    fGross.textContent = currencyNumber(gross);
    const fWeig = document.getElementById("weightedForecast"); if (fWeig)
    fWeig.textContent = currencyNumber(weighted);

    const gapEl = document.getElementById("goalGap"); if (gapEl) gapEl.textContent =
    weighted >= forecastGoal ? "Meta batida!" : currencyNumber(gap);
    const gapNote = document.getElementById("goalGapNote"); if (gapNote)
    gapNote.textContent = weighted >= forecastGoal ? "Forecast acima da meta" : "Falta
    faturar";
  }

```

```

const unvalEl = document.getElementById("dealsWithoutValue"); if (unvalEl)
unvalEl.textContent = `${dealsWithoutValue.length} sem valor`;

const summaryEl = document.getElementById("forecastSummary");
if (summaryEl) {
  summaryEl.innerHTML = `
    <div class="progress-row">
      <div class="progress-top">
        <span>Progresso do forecast contra a meta</span>
        <strong>${Math.round(goalProgress)}%</strong>
      </div>
      <div class="progress-track">
        <div class="progress-bar" style="width:${goalProgress}%;background:${weighted
>= forecastGoal ? "var(--success)" : "var(--primary)"}"></div>
      </div>
    </div>
    <div class="progress-row">
      <div class="progress-top">
        <span>Ticket médio comercial</span>
        <strong>${currencyNumber(dealsWithValue.length ? gross / dealsWithValue.length
: 0)}</strong>
      </div>
      <div class="conversion-meta">${dealsWithValue.length} oportunidade(s)
valoradas</div>
    </div>
  `;
}

const fStages = document.getElementById("forecastStages");
if (fStages) {
  fStages.innerHTML = stages.map((stage) => {
    if (!stage) return "";
    const stDeals = openDeals.filter((d) => d && d.stage === stage.name);
    const stGross = stDeals.reduce((sum, d) => sum + Number(d.value || 0), 0);
    const stWeig = stDeals.reduce((sum, d) => sum + Number(d.value || 0) *
(Number(d.probability || 0) / 100), 0);
    const impact = weighted ? Math.round((stWeig / weighted) * 100) : 0;

    return `
      <div class="conversion-row">
        <div class="conversion-top">
          <span>${escapeHTML(stage.name)}</span>
          <strong>${impact}% do forecast</strong>
        </div>
        <div class="progress-track">
          <div class="progress-bar" style="width:${impact}%;background:var(--info)"></div>
        </div>
      `;
  });
}

```

```

        <div class="conversion-meta">
            ${stDeals.length} negócio(s) · Pipeline: ${currencyNumber(stGross)} · Ponderado:
            ${currencyNumber(stWeig)}
        </div>
    </div>
    `;
    }).join("");
}

```

```

const fTable = document.getElementById("forecastTable");
if (fTable) {
    fTable.innerHTML = openDeals.length
        ? openDeals.map((d) => {
            if (!d) return "";
            const fore = Number(d.value || 0) * (Number(d.probability || 0) / 100);
            return `
                <tr>
                    <td>
                        <button class="link-button"
onclick="openDealDetail(${d.id})">${escapeHTML(d.company)}</button><br>
                        <small style="color:var(--muted)">${escapeHTML(d.contact || "Sem
contato")}</small>
                    </td>
                    <td>${escapeHTML(d.owner)}</td>
                    <td>${escapeHTML(d.stage)}</td>
                    <td>${currency(d.value)}</td>
                    <td>${d.probability}%</td>
                    <td><strong>${Number(d.value) > 0 ? currencyNumber(fore) :
"—"}</strong></td>
                </tr>
            `;
        }).join("")
        : `<tr><td colspan="6"><div class="empty">Nenhum negócio.</div></td></tr>`;
}
}

```

```

function activityMatchesTeamPeriod(a) {
    if (!a) return false;
    const period = document.getElementById("teamPeriod")?.value || "month";
    if (period === "all") return true;

    const actDate = new Date(a.date);
    if (period === "month") {
        return actDate.getMonth() === CURRENT_DATE.getMonth() && actDate.getFullYear()
=== CURRENT_DATE.getFullYear();
    }
}

```

```

const start = new Date(CURRENT_DATE);

```

```

start.setDate(CURRENT_DATE.getDate() - CURRENT_DATE.getDay());
start.setHours(0,0,0,0);
const end = new Date(start);
end.setDate(start.getDate() + 7);

return actDate >= start && actDate < end;
}

function renderTeam() {
  const openDeals = deals.filter((d) => d && d.status === "open");
  const pipeline = openDeals.reduce((sum, d) => sum + Number(d.value || 0), 0);

  const activeActivities = activities.filter(Boolean);
  const periodActivities = activeActivities.filter(activityMatchesTeamPeriod);
  const pending = periodActivities.filter((a) => getActivityStatus(a) === "Agendada").length;
  const overdue = periodActivities.filter((a) => getActivityStatus(a) === "Atrasada").length;

  const won = deals.filter((d) => d && d.status === "won").reduce((sum, d) => sum +
  Number(d.value || 0), 0);

  const tPipe = document.getElementById("teamPipeline"); if (tPipe) tPipe.textContent =
  currencyNumber(pipeline);
  const tPend = document.getElementById("teamPending"); if (tPend) tPend.textContent =
  pending;
  const tOver = document.getElementById("teamOverdue"); if (tOver) tOver.textContent =
  overdue;
  const tWon = document.getElementById("teamWon"); if (tWon) tWon.textContent =
  currencyNumber(won);

  const tRank = document.getElementById("teamRanking");
  if (tRank) {
    tRank.innerHTML = `
      <div class="team-row"
style="color:var(--muted);font-size:10px;font-weight:700;text-transform:uppercase">
        <div>Comercial</div>
        <div>Pipeline</div>
        <div>Atividades</div>
        <div>Vencidas</div>
        <div>Forecast</div>
      </div>
      ${owners.map((owner) => {
        const oDeals = openDeals.filter((d) => d && d.owner === owner);
        const oActs = periodActivities.filter((a) => a && a.owner === owner);
        const oOver = oActs.filter((a) => getActivityStatus(a) === "Atrasada").length;
        const oPipe = oDeals.reduce((sum, d) => sum + Number(d.value || 0), 0);
        const oForecast = oDeals.reduce((sum, d) => sum + Number(d.value || 0) *
        (Number(d.probability || 0) / 100), 0);

```

```

return `
  <div class="team-row">
    <div class="team-person">
      <span class="small-avatar">${owner ? owner.charAt(0) : "V"}</span>
      ${escapeHTML(owner)}
    </div>
    <div>${currencyNumber(oPipe)}</div>
    <div>${oActs.length}</div>
    <div>${oOver ? `<span
style="color:var(--danger);font-weight:700">${oOver}</span>` : "0"}</div>
    <div><strong>${currencyNumber(oForecast)}</strong></div>
  </div>
  `;
}).join("");
};
}

```

```

const alerts = periodActivities.filter((a) => getActivityStatus(a) === "Atrasada").sort((a,b)
=> new Date(a.date) - new Date(b.date));
const tAlerts = document.getElementById("teamAlerts");
if (tAlerts) {
  tAlerts.innerHTML = alerts.length
  ? alerts.map((a) => {
    const d = getDeal(a.dealId);
    return `
      <div class="activity-item">
        <div class="activity-icon">!</div>
        <div class="activity-info">
          <strong>${escapeHTML(a.title)}</strong>
          <small>${escapeHTML(a.owner)} · ${escapeHTML(d?.company || "Sem
negócio")}</small>
        </div>
        ${badgeForStatus("Atrasada")}
      </div>
    `;
  }).join("")
  : `<div class="empty">Tudo em dia na equipe!</div>`;
}
}

```

```

function renderReports() {
  renderConversionRows("reportsConversion");
  const losses = deals.filter((d) => d && (d.status === "lost" || d.status === "churn"));
  const reasons = {};

  losses.forEach((d) => {
    const reason = d.lossReason || "Não informado";
    reasons[reason] = (reasons[reason] || 0) + 1;
  });
}

```



```

});

const entries = Object.entries(reasons);
const max = Math.max(...entries.map(([, val]) => val), 1);

const lRep = document.getElementById("lossReports");
if (lRep) {
  lRep.innerHTML = entries.length
    ? entries.map(([reason, count]) => `
      <div class="progress-row">
        <div class="progress-top">
          <span>${escapeHTML(reason)}</span>
          <strong>${count}</strong>
        </div>
        <div class="progress-track">
          <div class="progress-bar" style="width:${(count / max) *
100}%;background:var(--danger)"></div>
        </div>
      </div>
    `).join("")
    : `<div class="empty">Sem perdas registradas.</div>`;
}

function fillStageOptions(selectedStage = (stages[0] ? stages[0].name : "")) {
  const select = document.getElementById("dealStage");
  if (!select) return;
  select.innerHTML = stages.map((s) => {
    if (!s) return "";
    return `<option value="${escapeHTML(s.name)}" ${s.name === selectedStage ?
"selected" : ""}>
      ${escapeHTML(s.name)} · ${s.probability}%
    </option>`;
  }).join("");
}

function openDealModal(dealId = null) {
  const form = document.getElementById("dealForm");
  if (!form) return;
  form.reset();

  const isEditing = Boolean(dealId);
  const deal = isEditing ? getDeal(dealId) : null;

  const editId = document.getElementById("editingDealId"); if (editId) editId.value =
deal?.id || "";
  const title = document.getElementById("dealModalTitle"); if (title) title.textContent =
isEditing ? "Editar negócio" : "Novo negócio";

```

```

fillOwnerOptions();
fillStageOptions(deal?.stage || (stages[0] ? stages[0].name : ""));

if (deal) {
    const comp = document.getElementById("dealCompany"); if (comp) comp.value =
deal.company || "";
    const cont = document.getElementById("dealContact"); if (cont) cont.value =
deal.contact || "";
    const phone = document.getElementById("dealPhone"); if (phone) phone.value =
deal.phone || "";
    const email = document.getElementById("dealEmail"); if (email) email.value =
deal.email || "";
    const val = document.getElementById("dealValue"); if (val) val.value = deal.value || "";
    const prob = document.getElementById("dealProbability"); if (prob) prob.value =
deal.probability || 60;
    const owner = document.getElementById("dealOwner"); if (owner) owner.value =
deal.owner || (owners[0] || "Victor");
    const close = document.getElementById("dealCloseDate"); if (close) close.value =
deal.closeDate || "";
    const source = document.getElementById("dealSource"); if (source) source.value =
deal.source || "";
    const tags = document.getElementById("dealTags"); if (tags) tags.value = deal.tags || "";
} else {
    const prob = document.getElementById("dealProbability"); if (prob) prob.value =
stages[0] ? stages[0].probability : 10;
    const owner = document.getElementById("dealOwner"); if (owner) owner.value =
owners[0] || "Victor";
}

const stageSelect = document.getElementById("dealStage");
if (stageSelect) {
    stageSelect.onchange = (e) => {
        const prob = document.getElementById("dealProbability");
        if (prob) prob.value = getStageData(e.target.value).probability;
    };
}

const modal = document.getElementById("dealModal");
if (modal) modal.classList.add("open");
}

function saveDeal(event) {
    event.preventDefault();

    const editingId = document.getElementById("editingDealId")?.value;
    const company = document.getElementById("dealCompany")?.value.trim();
    const contact = document.getElementById("dealContact")?.value.trim();

```

```

const phone = document.getElementById("dealPhone")?.value.trim();
const email = document.getElementById("dealEmail")?.value.trim();
const rawValue = document.getElementById("dealValue")?.value;
const value = (rawValue === "" || rawValue === undefined) ? null : Number(rawValue);
const probability = Number(document.getElementById("dealProbability")?.value || 60);
const stage = document.getElementById("dealStage")?.value;
const owner = document.getElementById("dealOwner")?.value;
const closeDate = document.getElementById("dealCloseDate")?.value;
const source = document.getElementById("dealSource")?.value;
const tags = document.getElementById("dealTags")?.value.trim();
const note = document.getElementById("dealNote")?.value.trim();

```

```

if (editingId) {
  const deal = getDeal(editingId);
  if (!deal) return;

  const oldStage = deal.stage;
  const oldOwner = deal.owner;
  const oldValue = deal.value;
  const oldProbability = deal.probability;

```

```

  Object.assign(deal, { company, contact, phone, email, value, probability, stage, owner,
closeDate, source, tags });

```

```

  addTimelineEvent(deal, "Negócio atualizado", "Alterações salvas.");
  if (oldStage !== stage) addTimelineEvent(deal, "Etapa alterada", `${oldStage} →
${stage}.`);
  if (oldOwner !== owner) addTimelineEvent(deal, "Responsável alterado", `${oldOwner}
→ ${owner}.`);
  if (Number(oldValue) !== Number(value)) addTimelineEvent(deal, "Valor alterado",
`Novo valor: ${currency(value)}.`);
  if (oldProbability !== probability) addTimelineEvent(deal, "Probabilidade alterada",
`${oldProbability}% → ${probability}%`);
  if (note) addTimelineEvent(deal, "Anotação adicional", note);
} else {
  const newDeal = {
    id: Date.now(), company, contact, phone, email, value, probability, stage, owner,
closeDate, source, tags,
    status: "open", createdAt: new Date().toISOString(),
    timeline: [
      { title: "Negócio criado", description: `Responsável inicial: ${owner}.`, date: new
Date().toISOString() },
      ...(note ? [{ title: "Nota inicial", description: note, date: new Date().toISOString() }] : [])
    ]
  };
  deals.push(newDeal);
}

```

```

saveData();
closeModal("dealModal");
refreshAll();

const detailsModal = document.getElementById("detailModal");
if (editingId && detailsModal && detailsModal.classList.contains("open")) {
  openDealDetail(Number(editingId));
} else if (!editingId) {
  showView("pipeline");
}
}

function openActivityModal(dealId = null) {
  const select = document.getElementById("activityDeal");
  if (!select) return;
  const form = document.querySelector("#activityModal form");
  if (form) form.reset();
  fillOwnerOptions();

  const activeDeals = deals.filter((d) => d && d.status === "open");
  select.innerHTML = activeDeals.length
    ? activeDeals.map((d) => `<option value="${d.id}" ${Number(d.id) === Number(dealId)
? "selected" : ""}>${escapeHTML(d.company)}</option>`).join("")
    : `<option value="">Nenhum negócio ativo</option>`;

  const selectedDeal = getDeal(dealId);
  const ownerInput = document.getElementById("activityOwner");
  if (ownerInput) ownerInput.value = selectedDeal?.owner || owners[0] || "Victor";

  const defaultDate = new Date(CURRENT_DATE);
  defaultDate.setHours(defaultDate.getHours() + 1);

  const dateInput = document.getElementById("activityDate");
  if (dateInput) {
    dateInput.value =
`${dateKey(defaultDate)}T${String(defaultDate.getHours()).padStart(2,
"0")}:${String(defaultDate.getMinutes()).padStart(2, "0")}`;
  }

  const modal = document.getElementById("activityModal");
  if (modal) modal.classList.add("open");
}

function createActivity(event) {
  event.preventDefault();
  const dealId = Number(document.getElementById("activityDeal")?.value);
  if (!dealId) return;

```

```

const newActivity = {
  id: Date.now(),
  title: document.getElementById("activityTitle")?.value.trim() || "Sem título",
  type: document.getElementById("activityType")?.value || "Tarefa",
  dealId,
  date: document.getElementById("activityDate")?.value,
  priority: document.getElementById("activityPriority")?.value || "Média",
  owner: document.getElementById("activityOwner")?.value || "Victor",
  description: document.getElementById("activityDescription")?.value.trim() || "",
  status: "Agendada"
};

activities.push(newActivity);
const deal = getDeal(dealId);
if (deal) addTimelineEvent(deal, `Ação criada: ${newActivity.title}`, `${newActivity.type}
programada para ${formatDateTime(newActivity.date)}.`);

saveData();
closeModal("activityModal");
refreshAll();

const detailsModal = document.getElementById("detailModal");
if (currentDetailDealId === dealId && detailsModal &&
detailsModal.classList.contains("open")) {
  openDealDetail(dealId);
}
}

function openDealDetail(dealId) {
  const deal = getDeal(dealId);
  if (!deal) return;

  currentDetailDealId = deal.id;
  const nextActivity = getNextActivity(deal.id);

  const header = document.getElementById("detailHeader");
  if (header) {
    header.innerHTML = `
      <div>
        ${badgeForStatus(dealStatusLabel(deal.status))}
        <span class="badge badge-gray">${escapeHTML(deal.stage)}</span>
        <h2 style="margin-top:10px">${escapeHTML(deal.company)}</h2>
        <p>${escapeHTML(deal.contact || "Sem contato")} · ${escapeHTML(deal.owner)} ·
        ${currency(deal.value)}</p>

      <!-- CHEVRON STAGE PROGRESS BAR INTERATIVO -->
      <div class="chevron-bar">
        ${stages.map((stage) => {

```

```

    if (!stage) return "";
    const isCurrent = stage.name === deal.stage;
    const currentIndex = stageNames().indexOf(deal.stage);
    const thisIndex = stageNames().indexOf(stage.name);
    const isCompleted = thisIndex < currentIndex;

    let cls = "";
    if (isCurrent) cls = "active";
    else if (isCompleted) cls = "completed";

    return `
    <div
      class="chevron-step ${cls}"
      title="Mover para: ${escapeHTML(stage.name)}"
      onclick="updateDealStageDirectly(${deal.id}, '${escapeHTML(stage.name)}')"
    >
      ${escapeHTML(stage.name)}
    </div>
    `;
  }).join("");
</div>
<div class="inline-actions">
  ${deal.status === "open" ? `<button class="button button-primary button-small"
onclick="openActivityModal(${deal.id})">+ Atividade</button>` : ""}
  <button class="close-button" onclick="closeModal('detailModal')">×</button>
</div>
`;
}

const detailsEl = document.getElementById("dealDetails");
if (detailsEl) {
  detailsEl.innerHTML = `
    <div class="detail-title">Informações</div>
    <div class="detail-value"><strong>Contato</strong><br>${escapeHTML(deal.contact
|| "Não informado")}</div>
    <div class="detail-value"><strong>Telefone</strong><br>${escapeHTML(deal.phone
|| "Não informado")}</div>
    <div class="detail-value"><strong>E-mail</strong><br>${escapeHTML(deal.email ||
"Não informado")}</div>
    <div class="detail-value"><strong>Valor
estimado</strong><br>${currency(deal.value)}</div>
    <div class="detail-value"><strong>Probabilidade
comercial</strong><br>${deal.probability}%</div>
    <div class="detail-value"><strong>Previsão de
Fechamento</strong><br>${deal.closeDate ? formatDate(deal.closeDate) : "Não
informada"}</div>
  `;
}

```

```

    <div class="detail-value"><strong>Próximo passo</strong><br>${nextActivity ?
escapeHTML(nextActivity.title) : "Sem atividades pendentes"}</div>
    <div class="detail-value"><strong>Origem do
Lead</strong><br>${escapeHTML(deal.source || "Não cadastrada")}</div>
    <div class="detail-value"><strong>Tags</strong><br>${escapeHTML(deal.tags ||
"Sem tags")}</div>

```

```

    ${deal.lossReason ? `<div class="detail-value"><strong>Motivo do
fracasso</strong><br>${escapeHTML(deal.lossReason)}</div>` : ""}

```

```

    ${deal.phone ? `
    <div class="whatsapp-box">
    <strong>WhatsApp Business</strong>
    <p>Envie mensagem rápida sem precisar salvar o número na agenda.</p>
    <button class="button button-success button-small"
onclick="openWhatsApp('${escapeHTML(deal.phone)}',
'${escapeHTML(deal.company)}')">● Abrir WhatsApp</button>
    </div>
    ` : ""}

```

```

    <div class="detail-title">Ações</div>
    <div class="detail-actions">
    ${deal.status === "open" ? `
    <button class="button button-secondary"
onclick="openDealModal(${deal.id})">Editar informações</button>
    <button class="button button-secondary"
onclick="openTransferModal(${deal.id})">Transferir responsável</button>
    <button class="button button-success" onclick="markDealWon(${deal.id})">Ganho
✓ </button>
    <button class="button button-danger" onclick="openLossModal(${deal.id})">Perdido
/ Churn ✗ </button>
    ` : `
    <button class="button button-secondary" onclick="reopenDeal(${deal.id})">Reabrir
oportunidade</button>
    `}
    <button class="button button-secondary" style="color:var(--danger);"
onclick="archiveDeal(${deal.id})">Deletar / Arquivar</button>
    </div>
    `;
}

```

```

const timelineEvents = [
  ...(deal.timeline || []),
  ...getDealActivities(deal.id).map((a) => ({
    title: `Atividade: ${a.title}`,
    description: `${a.type} · ${getActivityStatus(a)}${a.description ? ` · ${a.description}` : ""}`,
    date: a.date

```

```

    )))
].sort((a, b) => new Date(b.date) - new Date(a.date));

const timelineEl = document.getElementById("dealTimeline");
if (timelineEl) {
    timelineEl.innerHTML = `
        <div class="panel-header">
            <div>
                <h2>Timeline do Negócio</h2>
                <p class="panel-subtitle">Histórico de pontos de contato.</p>
            </div>
            <span class="badge badge-gray">${timelineEvents.length}</span> registros</div>
            <div>
                <div class="timeline-event">
                    <strong>${escapeHTML(e.title)}</strong>
                    <small>${escapeHTML(e.description)}</small>
                    <small style="color:var(--muted);">${formatDateTime(e.date)}</small>
                </div>
            </div>
        `
    timelineEl.innerHTML += `
        <div class="empty">Sem registros na timeline.</div>
    `;
}

const modal = document.getElementById("detailModal");
if (modal) modal.classList.add("open");
}

function openTransferModal(dealId) {
    const deal = getDeal(dealId);
    if (!deal) return;
    fillOwnerOptions();

    const trId = document.getElementById("transferDealId"); if (trId) trId.value = deal.id;
    const trOwner = document.getElementById("transferOwner"); if (trOwner) trOwner.value = deal.owner;

    const modal = document.getElementById("transferModal");
    if (modal) modal.classList.add("open");
}

function transferDeal() {
    const dealId = document.getElementById("transferDealId")?.value;
    const deal = getDeal(dealId);
    const newOwner = document.getElementById("transferOwner")?.value;

    if (!deal || !newOwner || deal.owner === newOwner) { closeModal("transferModal");
    return; }
}

```



```

const previousOwner = deal.owner;
deal.owner = newOwner;
addTimelineEvent(deal, "Negócio transferido", `${previousOwner} → ${newOwner}.`);

saveData();
closeModal("transferModal");
refreshAll();
openDealDetail(deal.id);
}

function markDealWon(dealId) {
  const deal = getDeal(dealId);
  if (!deal) return;
  if (!confirm(`Marcar "${deal.company}" como GANHO?`)) return;

  deal.status = "won";
  deal.probability = 100;
  addTimelineEvent(deal, "Oportunidade ganha", "Fechamento comercial realizado com sucesso!");

  saveData();
  closeModal("detailModal");
  refreshAll();
  showView("forecast");
}

function openLossModal(dealId) {
  const lossId = document.getElementById("lossDealId"); if (lossId) lossId.value = dealId;
  const modal = document.getElementById("lossModal"); if (modal)
modal.classList.add("open");
}

function markDealLost() {
  const dealId = document.getElementById("lossDealId")?.value;
  const deal = getDeal(dealId);
  if (!deal) return;

  const status = document.getElementById("lossStatus")?.value || "lost";
  const reason = document.getElementById("lossReason")?.value || "Não informado";

  deal.status = status;
  deal.lossReason = reason;
  addTimelineEvent(deal, status === "churn" ? "Churn computado" : "Negócio perdido",
`Motivação da quebra: ${reason}.`);

  saveData();
  closeModal("lossModal");
  closeModal("detailModal");
}

```

```

    refreshAll();
}

function reopenDeal(dealId) {
    const deal = getDeal(dealId);
    if (!deal) return;

    deal.status = "open";
    deal.lossReason = "";
    addTimelineEvent(deal, "Negócio reaberto", "Retornou ao funil de vendas ativo.");

    saveData();
    refreshAll();
    openDealDetail(dealId);
}

function archiveDeal(dealId) {
    const deal = getDeal(dealId);
    if (!deal) return;
    if (!confirm(`Remover definitivamente "${deal.company}"?`)) return;

    deal.status = "archived";
    addTimelineEvent(deal, "Negócio arquivado", "Removido do pipeline.");

    saveData();
    closeModal("detailModal");
    refreshAll();
}

function openWhatsApp(phone, company) {
    const cleanPhone = String(phone).replace(/D/g, "");
    const msg = encodeURIComponent(`Olá! Gostaria de alinhar o próximo passo comercial referente à sua oportunidade na empresa ${company}.`);
    window.open(`https://wa.me/55${cleanPhone}?text=${msg}`, "_blank");
}

function saveGoal() {
    const val = Number(document.getElementById("goalInput").value);
    if (isNaN(val) || val < 0) { alert("Meta incorreta."); return; }
    forecastGoal = val;
    saveData();
    refreshAll();
    alert("Meta mensal atualizada!");
}

function closeModal(id) {
    const modal = document.getElementById(id);
    if (modal) modal.classList.remove("open");
}

```

```
}

document.querySelectorAll(".modal-backdrop").forEach((backdrop) => {
  backdrop.addEventListener("click", (e) => {
    if (e.target === backdrop) backdrop.classList.remove("open");
  });
});

// STARTUP SEGURO
try {
  migrateOldDeals();
  saveData();
  fillOwnerOptions();
  renderCalendar();
  refreshAll();
} catch (e) {
  console.error("Erro crítico na inicialização automática:", e);
}
</script>
</body>
</html>
```